Highlights

cert-machine: independent exact certification of machine-generated mathematics—the method and three benchmark exercises[⋆]

Carlos Toledo

- Claims are certified, refuted with a counterexample, or refused with the reason.
- All 19 kissing-number bounds hold: 9.75×10^{15} pairs decided exactly.
- A constants registry removed four "unverified" marks after the replays.
- OpenAI's Navier–Stokes Lean lacked one clause of its theorem; the next commit adds it.
- The interval kernel shows no violation in 946,146 cases checked against Arb.

cert-machine: independent exact certification of machine-generated mathematics—the method and three benchmark exercises

Carlos Toledo[a,*]

[a]*Independent researcher, the cert-machine project, Brazil*

Abstract

Machines now produce mathematical claims—constants, constructions, bounds, formal proofs—faster than anyone can check them, and each arrives with a checker its producer wrote. We describe cert-machine, which decides such claims independently of their producers. A claim is read as a predicate over a declared set of inputs and receives one of three verdicts: certified, refuted with a counterexample, or refused with the reason. The decisions are computed in exact or outward-rounded interval arithmetic by code that shares nothing with the claimant, and every verdict is recorded with its evidence. Applied to three bodies of machine-generated claims, the system confirmed all 19 kissing-number bounds of an autonomous discovery system by deciding 9.75×10^{15} pairs exactly. It replayed the certificates behind four unverified bounds of an optimization-constants registry, which then removed the marks and cites the replays, and it refused a "certified" package whose evidence the package does not contain. It found that OpenAI's Lean development proves Clay's Navier–Stokes alternatives at the announced commit but not the energy clause of the paper's own theorem, which the next commit proves. The interval and exact verdicts are sound under the inclusion property of interval extensions; the double-precision interval kernel showed no violation in 946,146 cases checked against an independent library, and planted false, true and malformed claims were each decided correctly. Refusal is reported as a result.

⋆Draft v0.6 of 2026-10-19, built from repository commit 7c3b736d21ec; it will be archived with the next release of the repository, under the concept doi:10.5281/zenodo.22225860 (the previous version is in release v2026.10.18, doi:10.5281/zenodo.23446020). Every measured number and count in this manuscript is read at build time from the records named in the Data availability statement, and a record that no longer supports a sentence stops the build; the prose is authored and should be reviewed like any other. The code, the records and the readings were produced with AI agents under the author's direction (see the declaration). The author list and the CRediT roles are to be agreed with the prospective co-authors. Not peer-reviewed.

*Corresponding author.

Email address: carlos@carlostoledo.co (Carlos Toledo)

Keywords: machine-generated mathematics, independent checking,
interval arithmetic, exact arithmetic, certificates, verification, formal proof

1. Introduction

Machines now propose mathematics faster than anyone can read it. Program search finds constructions in combinatorics and analysis (Romera-Paredes et al., 2024; Novikov et al., 2025); autonomous research systems publish record configurations with their own verification files (Yang et al., 2026); conjectures about constants come from numerical coincidences (Raayoni et al., 2021); long proofs arrive with machine formalizations (OpenAI, 2026b,a). Each claim comes with a checker its producer wrote, and a producer's checker shares the producer's blind spots. The public record shows what that costs: a floating-point product printed as an exact constant; witnesses that hold only within a verifier's tolerance; a checker that compares in double precision what it encloses at a hundred digits; a pairwise condition proved by structure and never checked pair by pair; a clause of a theorem that the formal statement leaves out (Toledo, 2026d).

This paper proposes one rule for such claims and describes a system that enforces it. A machine-generated claim should be published with a bounded, reproducible evidence object that a checker independent of the claim's producer can decide; the checker must be able to certify the claim, refute it, or refuse it explicitly, and the scope and assumptions of the decision must be kept with it. cert-machine is such a checker. It reads a claim from its published statement and bytes, never runs the claimant's program, decides the claim in exact rational or outward-rounded interval arithmetic, and records one of three verdicts with the object that justifies it. When it cannot decide, it says so and why, and the refusal is recorded like any other result. Its semantics is sound under stated hypotheses, and the system is deliberately incomplete: a procedure that must always answer cannot be sound, since some of the questions machines ask are undecidable (Richardson, 1968), and a procedure that may refuse can be.

The contribution has three layers, which can be assessed separately (Table 1). The decision semantics defines what the three verdicts mean and proves the interval and exact ones sound under the inclusion property; Table 4 lists what each other pathway rests on. The certification implementations reduce particular kinds of claim to decisions of that kind, each stating what discharges a claim. The evidence infrastructure pins every input and every piece of code by digest, re-derives every record, and refuses to publish a number that no record supports. Interval arithmetic, the Krawczyk operator and exact geometric predicates are decades old (Moore et al., 2009; Krawczyk, 1969; Rump, 2010; Shewchuk, 1997), and nothing here claims

them; what is claimed is the semantics, its enforcement, and what the system decided.

layer	what it establishes	where	how it is checked
decision semantics	what certified, refuted and refused mean; their soundness under the inclusion property	Section 2.2	by reading the proofs
certification implementations	how each kind of claim is reduced to such decisions, and what discharges it	Sections 2.3–2.4, Appendix B	against an independent library, planted claims and second implementations (Section 4)
evidence infrastructure	inputs, code, witnesses, controls and history, pinned and re-derived	Section 2.5, Appendix A	by re-running the records

Table 1: The three layers of the contribution.

Section 2 gives the materials and the three layers. Section 3 applies them to three bodies of machine-generated claims: the kissing-number bounds of an autonomous discovery system, the certificates behind the unverified bounds of a registry of optimization constants, and OpenAI's mathematics release with its Navier–Stokes certificate. Section 4 evaluates the method itself as a decision procedure. Section 5 discusses what the exercises show, counts the independence the system can claim, places it beside existing practice, and states the threats to validity and the limits; Section 6 concludes.

2. Materials and methods

2.1. Materials

The claims. The system's inputs are claims and the bytes they rest on. A public register holds 132 claims—constants, constructions, bounds, formal statements, printed tables—from 59 claimants, with 3 rows generated here as calibrations, each pinned by the digest of the file that states it and decided from 25 records (Toledo, 2026d); of the exercises below it holds the Navier–Stokes row and the registry's three asterisked bounds. The three exercises of Section 3 draw on the following bodies of material. The kissing configurations of the Qiushi Engine (Yang et al., 2026): 19 point sets in dimensions 25, 27, 32–39, 43, 45 and 49–55, up to 53,301,140 points: 17 published as integer coordinates in three releases of the claimant's repository (Oxelra-AI, 2026), every file pinned by the digest the claimant's own manifest and the hosting platform report, and two, in dimensions 25 and 27, rebuilt here exactly from the construction files and closed forms in that repository. The optimization-constants registry (Tao et al., 2026) at commit 2c1968cd for the replays and with its pull-request queue as fetched on 2026-10-15, and at ec37689f and

46dbf036 after its two rounds of merges of 2026-10-18, each read the same day: the certificates its entries cite, among them three Zenodo deposits, a gist revision and a 104-file bundle, each pinned by digest, and the registry's own pages after the merges, pinned beside them. OpenAI's Navier–Stokes repository (OpenAI, 2026b) at its announced commit 8937a8f4 and its next, f9e8bc5b: 24 files pinned on 2026-09-09 (the two papers, the Clay statement, the Lean bridge files, the challenge statement, the announcement), and a Lean library of 2,486 files and 618,762 lines. OpenAI's mathematics release (OpenAI, 2026a) at commit fd4aeeb2: 719 manuscripts in 372 families and 416 Comparator challenges, censused file by file.

The machines. Everything that finishes within about a day runs on an 8-core laptop with 16 GB of memory: the interval kernel and the certifiers in Node v24.14.1 (V8 13.6), the bulk pair checks in Python with numpy's BLAS, the Lean builds. The 416 challenges of the release ran on GitHub's hosted runners (4 vCPU, 16 GB, six hours); the challenges those could not finish were sent to rented CPU machines under a pre-registered budget cap, and none of them finished there either, which the record says. The seven largest kissing configurations, about 1.4×10^{15} pairs each, ran on one rented NVIDIA L40S at $1.09 per hour: 6.8 GPU-hours of deciding, and $8.15 billed for the pod's whole life, its calibration and about 1.3 h idle after it finished included, by a kernel that was first required to reproduce the laptop's answers on two smaller dimensions. Every machine that decided a record is named in it with its platform and software.

The records. Every decision writes a record: the inputs by path and SHA-256, the verdict, the witness, the scope, the machine, the time, and the second implementation's reading where one exists. The records are the materials of Section 3 and of every number in this paper: each measured number and count is a macro written by a generator from the record that holds it, and the generator refuses to write when a record no longer supports the sentence that quotes it (Section 2.5).

Terms. Seven words recur. *Independent* means independent of the claim's producer: no code of the producer's is run or relied on; it does not mean that the system's own implementations are independent of each other (Section 5.2). A *pin* is a file named by path and SHA-256 digest and re-hashed whenever it is used. A *record* is the file a decision writes: the pinned inputs, the verdict, the witness, the scope and the machine. A *witness* is the object that justifies a verdict: a certificate, a counterexample or, for a split refusal, two proved points. A *battery* is a test program that exits with an error when any of its checks fails. A *red control* is a negative control in the sense of mutation testing (DeMillo et al., 1978; Jia and Harman, 2011): a deliberately wrong variant of a test, which must be rejected; a battery whose red controls do not all fire fails. A *reading* compares a statement with the

theorem it accompanies; it is recorded and issues no verdict. The verdict words are defined in Section 2.2 and listed in Table 2.

2.2. Decision semantics: the grammar

2.2.1. Claims over boxes

A *box* is a product $X = [\underline{x}_1, \overline{x}_1] \times \cdots \times [\underline{x}_n, \overline{x}_n]$ of closed intervals whose ends are rationals or infinite, with $\underline{x}_i < +\infty$ and $\overline{x}_i > -\infty$; a finite double is a rational, so a box of doubles is a box. A *declared envelope* B is a box each of whose sides carries the source of its bounds—a survey, a data sheet, a chart. A number someone printed is the box its digits allow: a printed 0.0634 is $[0.06335, 0.06345]$, and a fit printed to four decimals is a box in parameter space.

Definition 1 (interval extension)**.** Let $g : D \subseteq \mathbb{R}^n \to \mathbb{R}$. A map G from boxes in D to intervals is an *interval extension* of g if $g(X) \subseteq G(X)$ for every box $X \subseteq D$ (the inclusion property).

The inclusion property is the one hypothesis of every verdict below, and Section 2.3 is about securing it. The extensions used are natural extensions—the formula evaluated in interval arithmetic—and the classical theorem that a composition of operations with the inclusion property has it (Moore et al., 2009; Neumaier, 1990) carries it from the primitives to the model.

Definition 2 (claim)**.** A *claim* is a pair (B, φ) of a domain B—a box, or a finite set of rational points—and a predicate $\varphi(x) \equiv \bigwedge_j g_j(x) \le c_j$ whose functions have interval extensions G_j (an equality, or an exact predicate such as the orientation of three points, is the special case decided in exact arithmetic). It is read in one of two ways:

universal the claim asserts $\varphi(x)$ for every $x \in B$—the reading of a mathematical statement ("for all $n \le N$", "on $[0, 1]$", "for every member of this family", "for every pair of these points");

epistemic the claim asserts $\varphi(x^\dagger)$ for an unknown true input $x^\dagger \in B$—the reading of an engineering number, where B is the declared uncertainty about the input.

The two readings differ in what a single proved counterexample does. Under the universal reading it refutes the claim; under the epistemic reading it shows only that the declared evidence does not settle the question.

2.2.2. Three verdicts

Definition 3 (verdicts)**.** Let $\mathcal{X}$ be a finite cover of B by boxes (a finite set is covered by its thin boxes, its points). The claim is

CERTIFIED if $\sup G_j(X) \le c_j$ for every j and every $X \in \mathcal{X}$;

REFUTED under the universal reading, if some box $X^* \subseteq B$ (typically a thin box, a point) has $\inf G_j(X^*) > c_j$ for some j; under the epistemic reading, if every $X \in \mathcal{X}$ has $\inf G_j(X) > c_j$ for some j, or some g_j has a proved lower bound $d_j > c_j$ on all of B;

REFUSED otherwise. Under the epistemic reading a refusal is *split* when two points $x^-, x^+ \in B$ have been proved to satisfy and to violate φ respectively, and *unresolved* when the enclosures did not separate within the budget.

Proposition 1 (soundness)**.** *If every G_j is an interval extension of g_j, then* CERTIFIED *implies $\varphi(x)$ for every $x \in B$;* REFUTED *implies, under the universal reading, $\neg\varphi(x)$ for some $x \in B$ and, under the epistemic reading, $\neg\varphi(x)$ for every $x \in B$; and a split* REFUSED *implies that neither $\forall x \in B\ \varphi(x)$ nor $\forall x \in B\ \neg\varphi(x)$ holds, so that $\varphi(x^\dagger)$ is not determined by the declared envelope.*

Proof. Let $x \in B$ and choose $X \in \mathcal{X}$ containing it. By the inclusion property $g_j(x) \in g_j(X) \subseteq G_j(X)$, hence $g_j(x) \leq \sup G_j(X) \leq c_j$ for every j, which is $\varphi(x)$. The refutations are the same argument with the inequality reversed, on X^*, on every member of the cover, or from the proved bound $g_j \geq d_j > c_j$ on B. For a split refusal, x^- and x^+ are points of B at which φ is proved true and false respectively; either could be $x^\dagger$. □

An unresolved refusal claims nothing, and the grammar keeps it that way: a refusal is terminal, never retried at lower rigour, never converted into a probability and never dropped from the record (Toledo, 2026d). The only path from REFUSED to CERTIFIED is a new exact argument—a finer cover, a better extension, a different certifier, or, as Section 3.1 shows, the bytes the claimant had not yet published. The three words are not probabilities and there is no fourth.

2.2.3. Witnesses

Every verdict carries the object that justifies it, and the object is what is published. A certification carries its *certificate*: the inputs pinned by digest, the cover, the enclosures, and a statement of what would make it fail. The certificate constructor of the interval toolkit refuses to build a certificate whose falsifier is not stated and a CERTIFIED that carries no evidence, on the principle that a certificate that cannot go red is decoration. A refutation carries the falsifying object, printed—a counterexample, an observation outside a region, a violated equation, a digit that is not the rounding of an enclosure. A split refusal carries its two witnesses. An exact certificate—integers and rationals—can be re-checked without any of this system's code; an interval certificate can be re-checked by any arithmetic that rounds outward at the precision it states.

2.2.4. Thresholds

A split refusal is the most useful outcome the grammar has, because the question it leaves open has a size. When the envelope depends on a declared scalar p—an operating choice that moves the box, or one end of an interval that a measurement could narrow—the procedure reports the value of p at which the verdict would change, each endpoint decided like any other claim, and a refusal becomes a measurement or an operating change with a known effect on the verdict. A moving box has two such values and a shrinking one has one; the definitions, the proposition that makes them sound and the engineering demonstrator's two uses (Toledo, 2026c) are in Appendix C. None of the three exercises needs them.

2.2.5. Compositions

A receipt with several checks is REFUTED if any check is refuted, otherwise REFUSED if any is refused, otherwise CERTIFIED; by Proposition 1 this is sound for their conjunction. The public register composes the three verdicts into words that each say what they are made of (Table 2); none is a fourth verdict. Two consequences are enforced rather than recommended (Toledo, 2026d). A scope sentence is load-bearing, because a CERTIFIED without the scope beside it can say more than the record, as when the computational fragment of a claim is certified and its analytic core is not touched. And the kinds of refusal are never summed: the instrument declining, the claimant having published nothing, a budget exhausted, a cell whose box did not close and an audit stating its reach are different behaviours with different denominators.

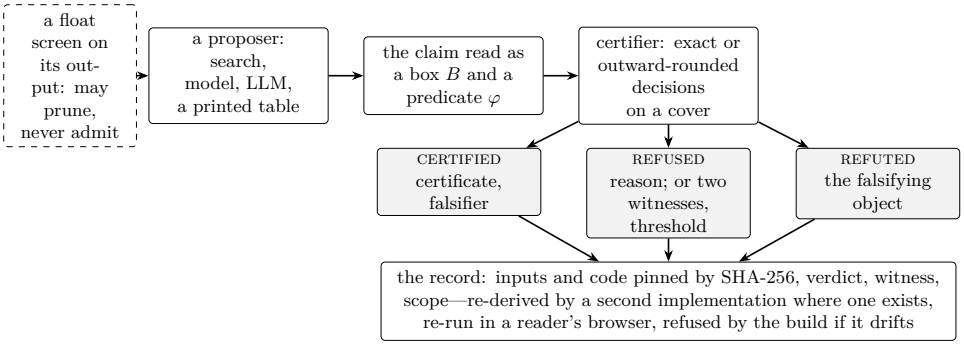


Figure 1: The system. A proposer is never opened and never trusted; its output is read as a claim over a box and decided by a certifier; the verdict, its witness and the pinned inputs form a record; an exact certificate can be re-checked without the system's code.

2.3. Certification implementations: the arithmetic

Proposition 1 has one hypothesis, the inclusion property, and the arithmetic exists to secure it. Two kinds are used, and the choice is made per

word	composed of	the witness the row carries
CERTIFIED	—	a certificate with its falsifier; an exact one re-checkable without the system's code
REFUTED	—	the falsifying object, printed
REFUSED	—	the reason; for a split refusal, two witnesses and the thresholds
PARTIAL	CERTIFIED + REFUSED	which fragment certified and which core was not reached
NEEDS DATA	REFUSED, the cause with the claimant	the bytes that would decide the row, named
MIXED	CERTIFIED + REFUTED, one record	the counts of both, and the items refuted
REPAIRED	REFUTED + CERTIFIED	the object as printed refuted; the object beside it certified
QUEUED	not a verdict	nothing; the row is not counted as decided

Table 2: The verdicts and their compositions in the public register (Toledo, 2026d). Some surfaces display a CERTIFIED under another word, and none is a further verdict: engineering pages write PROVED; the register writes WITNESSED for a configuration whose published bytes are themselves the exact object decided; a ranking of families says DECIDED; the words for a printed number—REPRODUCED, CONSISTENT, OFF THE MAXIMUM, OUTSIDE ITS SUPPORT—refine the three for a comparison of two boxes (Toledo, 2026b).

quantity: exact rationals where a tie must be decided or the object is rational, outward-rounded intervals where the quantity is transcendental or the data are long.

Exact rationals. Numbers are reduced fractions of arbitrary-precision integers. Doubles convert to them losslessly; linear systems are solved by exact elimination, which reports a singular matrix rather than a small pivot; conversion back to a double is for display only. Rational arithmetic decides every sign it is asked, and it is used for the corners of multilinear forms (Section 2.4.3), the expected counts of a benchmark, polynomial identities, and the ends of enclosures whose comparison must be exact. Decimal literals—the digits a claimant printed—are read as the rationals they denote and kept as strings beside their double images, so that a vertex printed to three decimals and an observation printed to four can be found to coincide (Toledo, 2026b).

Exact integers in floating-point hardware. A product of integer vectors is itself an integer, and floating-point hardware computes it exactly when no partial sum can leave the range the format holds exactly: 2^{24} for single precision, 2^{53} for double, 2^{11} for the half-precision inputs of tensor cores, which accumulate in single precision (Fasi et al., 2021). The bound used

is the Cauchy–Schwarz one—every partial sum of products of two vectors is at most the product of their lengths—checked against the data before anything runs and written into the record. This is what lets 10^{15} pairs of integer points be decided exactly on a GPU (Section 3.1): the arithmetic is floating point, the result is an integer held exactly, and the test applied to it is the exact integer test.

Outward rounding on doubles. IEEE-754 arithmetic in round-to-nearest returns, for each of $+, -, \times, \div$, the double nearest the exact result, which therefore lies within half a unit in the last place of the computed one (IEEE, 2019; Goldberg, 1991). JavaScript cannot set the rounding mode, so directed rounding is emulated: each computed endpoint is moved one unit in the last place outward by stepping its bit pattern (Rump et al., 2009). The trust base of this emulation is stated once: the correctly rounded $+$, $-$, $\times$, $\div$ and square root that IEEE-754 mandates and V8 implements, V8's arbitrary-precision integers, and nothing in `Math.*`, which ECMA-262 leaves "implementation-approximated" (a 2017 proposal to require one-ulp accuracy was closed without adoption); the Node version that produced every record is written into the record (`v24.14.1`, V8 13.6, whose `Math.*` is the fdlibm port, faithful to within one ulp and not correctly rounded; V8 moved these functions to a correctly rounded library in May 2026, which is why the version is material; correctly rounded libraries are a recent engineering achievement (Sibidanov et al., 2022)). The operations are those of the basic set of the interval standard (IEEE, 2018); the implementation is not a conforming one and carries no decorations.

Proposition 2. *Let $\circ \in \{+, -, \times, \div\}$, let A, B be intervals of doubles—pairs of non-NaN doubles with lower end at most upper end, an infinite endpoint allowed as a bound—with $0 \notin B$ when $\circ = \div$, and let ℓ and h be the round-to-nearest values of the smallest and largest of the endpoint combinations $a \circ b$. If no endpoint combination is indeterminate ($0 \cdot \infty$, $\infty - \infty$, ∞/∞), then $A \circ B \subseteq [\mathrm{down}(\ell), \mathrm{up}(h)]$, where* down *and* up *step one double toward $\mp\infty$ and fix $\mp\infty$; an exact result beyond the largest double rounds to $\pm\infty$, which the step fixes, so overflow is enclosed.*

Proof. For these operations the extremes of $a \circ b$ over a product of intervals are attained at endpoint combinations (monotonicity in each argument on each sign region, with $0 \notin B$ for division). The exact extreme lies within half the spacing of the doubles on its side of its round-to-nearest value, and the adjacent double on that side is a whole spacing away (subnormal spacing included), so it lies on the correct side of $\mathrm{down}(\ell)$ and $\mathrm{up}(h)$. □

The domain of the proposition is enforced, not assumed: a malformed operand (a NaN, an inverted pair) or an indeterminate result is refused by exception and never decided, because a consumer written as "not proved

above the bar, hence below it" would otherwise certify on a NaN (until 2026-10-09 nothing checked this, and the batteries skipped non-finite results; enforcing it exposed one battery check that had been passing on malformed boxes). Division by an interval that contains zero is refused, not extended; a square is clamped at zero; a non-integer power is refused, and so is an integer exponent of 2^{31} or more, where the implementation's exponent arithmetic would wrap. The widening costs about one bit per operation and nothing at the accuracy the claims need; the domain checks cost a few comparisons per operation. Where more precision is required—a constant to be certified to twelve digits, a Catalan or Li_2 value—a separate arbitrary-precision arithmetic carries a binary mantissa of arbitrary length with true directed rounding (floor and ceiling of the shifted integer), and converts its results to doubles outward with an exact check.

The elementary and special functions, long sums and derivatives that the certifiers also need are built on these operations; how each is enclosed, and the margins that make the series tails safe, are in Appendix B.

2.4. Certification implementations: the certifiers

Each certifier below turns a question that is analytic in form into finitely many decisions of the kind Proposition 1 covers. None is new; the assembly is, together with the rule that each refuses rather than guesses. Table 3 lists them with the obligation each discharges and the kind of assurance it gives, so that a shared verdict word is not read as a shared level of assurance, and Table 4 states what each pathway's verdicts rest on beyond Proposition 1.

2.4.1. An entropy certificate

A sums–differences constant is bounded below by any finite law μ on $\mathbb{Z}^2$, through a ratio of Shannon entropies of linear forms of $(X, Y) \sim \mu$ (Section 3.2 gives the ratio). The certifier computes the law of each form $L(X, Y)$ exactly, as the pushforward of μ in rationals, and encloses each term $-p \ln p$ in an interval, the logarithm by its series with an outward-rounded tail; each entropy is a sum of these enclosures and the ratio is a quotient of intervals. One implementation computes in dyadic arbitrary-precision intervals at 128 bits, the other in Python's `decimal` at 120 digits rounded outward (Table 5); neither is the double-precision kernel that Section 4 compares with Arb. A mixture of laws over the digits of an expansion adds the per-law entropies only when the map from digits to $L(X, Y)$ is injective, a condition checked from the supports.

2.4.2. A zero, exactly one

Theorem 1 (Krawczyk; see Krawczyk, 1969; Neumaier, 1990; Rump, 2010)**.** *Let $f : D \subseteq \mathbb{R}^n \to \mathbb{R}^n$ be continuously differentiable, $X \subseteq D$ a box, $\hat{x} \in X$, Y a nonsingular matrix and $F'(X)$ an interval extension of the Jacobian*

certifier	what discharges the claim	arithmetic	kind of assurance
a finite box, exhaustively	every element decided; a bound that makes floating point exact, checked first	exact integers	computation
an entropy certificate	exact pushforwards; each $-p \ln p$ enclosed	rationals and intervals	computation
a zero, exactly one	the Krawczyk inclusion (Theorem 1)	intervals	computation
a maximum in a box	one zero of the gradient and a negative-definite Hessian over the box (Proposition 3)	intervals	computation, local to the box
a supremum at a limit	a score of one sign over the box (Proposition 4)	intervals	computation
a statement for every input	a cover, a monotone bracket, or the vertices of a multilinear form (Lemma 1)	intervals or rationals	computation
a sign	a forward-error bound, else exact integers	doubles, then integers	computation
a polynomial, nonnegative	an exact sum-of-squares identity	rationals	computation
a minimum of a cosine polynomial	a Sturm count and V. Markov's bound	rationals and intervals	computation
a formal proof, built	the kernel's acceptance, replayed by a second kernel	the kernels' own	computation, by kernels others wrote
a formal statement, read	none: the statement is compared with the theorem it accompanies	—	a reading (AI agents), no verdict

Table 3: The certification pathways, each with what discharges a claim and the kind of assurance that gives. A reading is listed because three results of this paper rest on one in part (Section 5.5); it never issues a verdict.

over X. If

$$K(X) = \hat{x} - Yf(\hat{x}) + \big(I - YF'(X)\big)(X - \hat{x}) \subseteq \operatorname{int} X,$$

then f has exactly one zero in X.

The certifier takes $\hat{x}$ from a floating-point search, Y as an approximate inverse of the Jacobian at $\hat{x}$, evaluates $K(X)$ with every datum enclosed by its neighbouring doubles, and inflates X for a bounded number of rounds; if the inclusion never holds the zero is REFUSED with the reason "no contraction". A sibling certifier for fixed points in weighted sequence spaces uses the radii-polynomial form of the same argument and requires the contraction factor to be proved below one before it states uniqueness.

pathway	its verdicts rest on	checked by
interval decisions	the inclusion property, secured by the one-ulp outward step (Proposition 2) over the correctly rounded basic operations of IEEE-754 as V8 implements them	the comparison with Arb (Section 4); red controls
exact rationals and integers	the arbitrary-precision integers of V8 and of Python	second implementations
a finite box in floating point	the Cauchy–Schwarz bound, checked against the data, and the hardware's exact integer products below it: BLAS in single precision; half-precision tensor-core inputs accumulated in single precision (Fasi et al., 2021)	a calibration against the laptop's counts; seeded samples in exact integers
entropy certificates	dyadic arbitrary-precision intervals at 128 bits; Python's `decimal` at 120 digits, rounded outward	two implementations; the comparison with Arb (Section 4)
the entropy–influence bound	the integer Walsh–Hadamard transform, `decimal` logarithms, and an amplification theorem that is cited (O'Donnell and Tan, 2013; Hod, 2017)	10 named checks; a second implementation (Table 5); the comparison with Arb
formal proofs	the Lean kernel and a second kernel, the export of the proof, Comparator's judging of the challenge statement, and the library build the claimant pinned	the second kernel; the statement is read, not decided
every pathway	decoding the claimant's bytes, and the reading of the claim as (B, φ)	the pins; the scope sentence

Table 4: What each pathway's verdicts rest on beyond Proposition 1, and what checks it. The first two rows are the setting of the proposition; each other row adds what it lists.

2.4.3. A statement for every input in a box

Two reductions decide a universal statement over a box in the exercises; two more, for monotone implicit functions and multilinear forms, are in Appendix B.

A finite box, exhaustively. Where the box is a finite set—the pairs of a point configuration, the cases of a theorem up to a bound—every element is decided, and the only question is the arithmetic: exact integers, or floating point under a bound that makes it exact (Section 2.3). The work is proportional to the set, and a set too large for the budget is a refusal, never a sample.

Bisection with a proposer. The box is split until each piece's enclosure decides, a floating-point heuristic choosing the coordinate to split and never a verdict; a depth budget bounds the work, and an exhausted budget is an unresolved refusal. To publish a split refusal, the corners are evaluated as thin boxes and one proved witness is kept on each side.

The other certifiers of Table 3—a maximum in a box, a supremum at a limit, monotone implicit functions, multilinear forms, an exact sign, a

nonnegative polynomial and the minimum of a cosine polynomial—play no part in the exercises and are described in Appendix B.

2.4.4. A formal proof, built and read

A formal certificate is two claims. That the proof is accepted is decided by building it at the pinned commit, with the toolchain and library versions the claimant pinned, and asking the kernel which axioms the result depends on; where the claimant's checker allows it, the proof is replayed by a second kernel written independently of the first (Lean FRO, 2026; ammkrn, 2026). That the statement proved is the one claimed is not a computation: the statement is read hypothesis by hypothesis against the published theorem and the library's definitions, the reading is recorded with its slips and weak points, and it never issues a verdict of its own—the verdict is the kernel's, scoped by what the reading found absent. Section 3.3 is the worked case. The readings reported in this paper were produced by AI agents working from a written brief and consolidated into the records cited (Section 5.5); they are recorded as readings and carry no verdict.

2.4.5. Proposers

Every certifier above takes its candidate from a floating-point search—Nelder–Mead and Levenberg–Marquardt for a likelihood, Newton for a root, sampling for a minimum, a semidefinite solver for a decomposition, a language model for a construction—and none of these is ever relied on for a verdict. A proposer may be wrong, slow or adversarial; it can cost a refusal, never a false verdict.

2.5. Evidence infrastructure: the verification discipline

Proposition 1 makes the verdicts as sound as the code that computes the extensions. The discipline is how the system guards that code. Its rules are stated as engineering because each is a line of code that refuses—a battery that exits non-zero, a page builder that will not write its page, a numbers generator that will not write a paper's macros—and this section says where each refusal sits.

Screens prune, never admit. Floating point decides only what is worth certifying. In the generation loop, which proposes objects of the system's own, every family declares a cheap screen that may only discard candidates and a certifier that is the only authority, and the campaign runner plants known answers that must survive every stage before a run may start (Toledo, 2026a, the methods note).

Red controls must fire. A check that has never gone red is decoration. A red control is a deliberately wrong variant of a test that must be rejected, and a battery whose red controls do not all fire fails, whatever its other checks say.

Examples: a determinant whose double-precision value is $+3.6 \times 10^{-15}$ and whose exact sign is negative, which the filter must send to exact arithmetic; a "gate" that evaluates the floating-point model at the centre of the box, which is what a point pipeline does and which the corners must catch; a missing inner certificate, which must refuse a theorem; a bar on the wrong side of an endpoint, which must be refused by name; a seventh point of equal norm in the plane, which the pairwise test must find. The test target runs 147 batteries, 93 in JavaScript and 54 in Python, and a wiring check refuses a tree in which that target and the control build's list differ. In the six-lane audit of machine-written manuscripts, 21 mutation controls stand beside 92 named checks, and the campaign runner ships 19 ways to cheat its own reward signal, each caught by a named control (Toledo, 2026d). The methods note records 16 defects the project shipped and then found: 10 caught by an impossible number, a calibration, a control, a byte pin or an outside read, and 6 by the adversarial read of the code made for this paper (Section 5.5), counted apart (Toledo, 2026a, the methods note).

Calibrate before you decide. A new path—a new machine, a new arithmetic, a new kernel—decides nothing until it has reproduced a decision already made by an old one. The GPU kernel of Section 3.1 was required to return the laptop's exact contact counts on two smaller configurations before it was allowed to decide the seven large ones; the Leech lattice's 196,560 and E_8's 240 are re-derived at the start of every kissing run; and a pre-registration states, before a paid run, what will be measured, what counts as a result, and at what cost the run stops.

Pin the bytes. Every claim is decided against a held byte sequence named by path and SHA-256 and re-hashed at decision time: the claimant's file, the dataset, the printed table transcribed digit for digit, and the certifying modules themselves, whose digests a ledger records and whose workers refuse other bytes. A page that reports a record refuses to build when a pinned input has changed, when the record was written by a different version of its verifier, or when its battery does not pass; an application build that regenerates a record and finds it different from the committed one refuses and leaves the committed record in place. Files lifted from the laboratory this system grew out of come through a manifest of their source paths and digests.

Independence from the claimant. The claimant's program is never relied on: the decision is re-derived from the published statement and the published bytes, and where a claimant's script was read to settle which identity a manuscript meant, the record says so and the script was not executed. An unqualified "no shared code" would not survive a check, because two of the system's own instruments reuse its own code across a producer–checker line: the mean-field-game capacity certifier (`instruments/mfgcap`) imports

the frozen verifier of an earlier published report and extends it, and the lemniscate certifier (`instruments/lemniscate`) shares the interval kernel with the bench it was lifted from; the precise form adopted is that the system *shares no code with the claimant*, and every certificate re-runs with no engine present (Toledo, 2026a, the positioning notes).

Same-author second implementations. Where a verdict is the product, a second program written from the specification, in another language and arithmetic and sharing no code, re-derives it (Table 5). These are N-version checks by the same author from the same specification on the same inputs, written by AI agents from one brief—the correlated-failure case that Knight and Leveson (1986) measured: they catch a slip in one arithmetic and cannot catch a shared misreading, and they are counted apart from re-derivations by other parties (Section 5.2), of which the record holds 1. Not every certifier has one; Section 5.5 names those that do not.

Records re-run. A record is not believed because it was written; it is re-derived. Published records re-run in the reader's browser by the bytes that wrote them and are compared with the record—by digest for a day of wave-window decisions and a cell of the extreme-value atlas, field by field for an engineering receipt and an abatement registry. A rerun kit lists 31 commands that re-derive the register's records or verify them with the standard library alone; its one recorded execution, on 2026-10-02, before the exercises of Section 3, ran 23 of them in about 22 minutes on a laptop and compared the 88 register rows it then held: 0 moved, 0 failed. The exercises' records are re-read against their pins at every build, and the registry's and the evaluation's are re-derived by their batteries; the kissing pairs, which took GPU-hours, are not re-derived at build. And the system's own results are offered to outside parties to re-certify without its code; 1 such rerun is recorded so far, the $\lambda(4)$ audit of Section 3.4.

Honest counting. Every row is derived from a record; a claim with no record gets no row. An aggregate audit counts once and names its most frequent defect. A correction the system adds beside a refuted claim is never counted among the audited corpus's rows; a queued row is not a decided row; refusal kinds are never merged into one total; and every number in this paper and its companions is a macro written by a generator that refuses when a record no longer supports the sentence that quotes it (Toledo, 2026d).

decision	the two implementations	recorded agreement
kissing pairs	numpy or a GPU kernel on integer rows; plain Python integers and a BigInt JavaScript reading on seeded samples	every sample's contact count equal in all three, in all 19 configurations
entropy certificates	JavaScript over BigInt with dyadic intervals at 128 bits; the Python standard library's `decimal` at 120 digits	both CERTIFIED on the registry's three entropy certificates; the two-law ratio's enclosures sharing 31 digits, the width of the JavaScript one
the entropy–influence bound	Python `decimal` intervals at 60 digits; JavaScript over BigInt, the entropy in closed form with 4 logarithms enclosed	both CERTIFIED, the JavaScript enclosure inside the Python one and around an Arb ball; 12 planted claims of known truth decided as expected by both
step functions	JavaScript over BigInt with Kronecker products; Python `fractions`, every lag summed term by term	the same exact fraction in both on all 7 certificates of C_{1a}–C_{1d}
the trace-constant certificate	JavaScript over BigInt rationals and the interval kernel; Python `fractions` with its own outward-rounded logarithm	both CERTIFIED; the density proved positive on 3,658 boxes in each, 6 red controls firing in both
δ_3 certificates	the claimant's Python verifier; a clean-room JavaScript verifier	5 of 5 certificates verified by both (Toledo, 2026e)
a Hankel-determinant wall	a lifted arbitrary-precision engine; a standard-library cross-check	51 rows re-run bit-identical, 6 of 6 cross-checks agreeing
three engineering decisions	a gate, an abatement kernel and a day of wave-window decisions, each in JavaScript intervals or rationals and again in Python (`decimal`, `fractions`, a clean-room verifier)	agreement recorded in each (Toledo, 2026d)

Table 5: Decisions re-derived by a second implementation that shares no code with the first—written by the same author, with AI agents, from the same specification, so an N-version check rather than an independent re-derivation. Each agreement is read from the record that holds it.

3. Benchmark exercises

Three exercises test the method on claims that machines produced, each a different kind of claim with a different outcome: a configuration of points, decided pair by pair; the certificates behind a registry's unverified bounds; and the formal proofs and challenges of one release. Each is told in the same order: the claim, how it is read, how it is decided, the verdict and its witness, the cost, and what the claimant did next. Every number is read from the record that decided it.

3.1. A configuration of points: nineteen kissing-number bounds from an autonomous discovery system

The claim. In September 2026 a group at Zhejiang University published new lower bounds for the kissing number (Conway and Sloane, 1999) in 19 dimensions (25, 27, 32–39, 43, 45 and 49–55), found by an autonomous multi-agent research system (Yang et al., 2026). Each bound is a configuration of unit vectors, no two closer than sixty degrees, larger than the bound the claimant cites as previous. The configurations are built from lattices, codes, sections and projections. Their sizes are proved by moment identities and the angular condition by structure, and the release's own verification files record that the pairwise inner products were not checked.

The reading. A kissing configuration is the simplest universal claim there is: a finite set of points and one inequality for every pair. In the terms of Definition 2, the box is the set of pairs and the predicate is $\langle x, y\rangle \le \frac{1}{2}|x|\,|y|$. With integer coordinates the test is exact. For rows of squared norms A and B with inner product p, the angle is at least sixty degrees if and only if $p \le 0$ or $4p^2 \le AB$; the same test rejects a repeated or parallel row, so it also checks that the directions are distinct, and a zero row is rejected separately, by its norm. A contact, at exactly sixty degrees, is $4p^2 = AB$.

The decision. Two configurations, in dimensions 25 and 27, were published with the paper as exact objects and were decided at once. The other seventeen had no published points, so they were marked NEEDS DATA, with the missing files named. We asked for them on the repository's public thread. The claimant published the points for dimensions 43 and 45 five days later and, once those were decided, every remaining configuration within hours (Oxelra-AI, 2026). Each file is pinned by the digest that the claimant's manifest and the hosting platform report, and the tool refuses a file that does not match.

For every configuration, the row count and the counts per squared norm are checked against the claimant's, the rows are shown to span exactly d dimensions (the rank over $\mathbb{Q}$ of the exact Gram matrix XX^{T} of the rows), and every pair is decided. Three arithmetics are used, each exact on its data. On the laptop, single-precision BLAS tiles are exact because every

partial sum is bounded by the largest squared norm (at most 4,608, in dimension 25, against 2^{24}). In dimensions 25 and 27 the coordinates are, after scaling, integer combinations of $1, \sqrt{2}, \sqrt{3}, \sqrt{6}$ (the claimant's closed forms, such as $\sqrt{3}/2 + \sqrt{2}/4$), so each inner product is such a combination with integer coefficients, and its sign is decided exactly; in dimension 25, 1,118,436 products are computed in exact integers outside BLAS. In dimension 38, where 1,932 rows have coordinates of 36-digit squared norm, those rows are decided in exact 64-bit integers (every partial sum is below $\sqrt{48N} < 2^{63}$, where $N \approx 3.95 \times 10^{35}$ is their squared norm and 48 that of the other rows) and among themselves in Python integers. In dimensions 49–55, about 1.4×10^{15} pairs each, a fused kernel on one rented GPU computes and reduces each 128×128 block of products in registers, with half-precision inputs and single-precision accumulation; every value is an integer below 2^{11}. Before deciding anything, the GPU path had to reproduce the laptop's contact counts on dimensions 32 and 39 (358,430,976 and 325,490,632), and it did. In every dimension a seeded sample of rows was decided again in plain Python integers and by a separate JavaScript implementation in arbitrary-precision integers; the three agree on every contact count. Dimensions 43 and 45 were also decided in the form the construction states—integer coefficients over the claimant's lattice basis, with its exact Gram matrix—with the same contacts.

d	points	pairs decided	at exactly 60°	arithmetic	time
25	197,580	19,518,829,410	446,074,805	float32 BLAS + exact	37 s
27	201,567	20,314,526,961	444,139,680	float32 BLAS	27 s
32	347,584	60,407,144,736	358,430,976	float32 BLAS	45 s
33	363,968	66,236,170,528	368,310,912	float32 BLAS	52 s
34	384,196	73,803,091,110	380,481,792	float32 BLAS	59 s
35	409,676	83,917,007,650	315,579,800	float32 BLAS	1 min
36	484,760	117,495,886,420	404,388,832	float32 BLAS	1 min
37	498,024	124,013,703,276	392,397,168	float32 BLAS	1 min
38	591,900	175,172,509,050	379,853,270	float32 BLAS, exact int64, integers	3 min
39	763,668	291,594,025,278	325,490,632	float32 BLAS	3 min
43	2,553,792	3,260,925,512,736	4,467,331,098	float32 BLAS	40 min
45	7,380,090	27,232,860,514,005	19,832,543,736	float32 BLAS	9.1 h
49	52,430,156	1,374,460,602,877,090	965,197,554,294	fused GPU kernel	53 min
50	52,458,468	1,375,945,406,224,278	964,169,164,800	fused GPU kernel	53 min
51	52,500,930	1,378,173,799,181,985	962,629,120,340	fused GPU kernel	53 min
52	52,585,772	1,382,631,682,125,106	959,560,825,568	fused GPU kernel	53 min
53	52,698,696	1,388,576,253,700,860	955,491,412,444	fused GPU kernel	53 min
54	52,923,774	1,400,462,900,739,651	947,433,469,328	fused GPU kernel	1.2 h
55	53,301,140	1,420,505,735,999,230	934,069,080,624	fused GPU kernel	1.2 h

Table 6: The 19 configurations of Yang et al. (2026), every pair decided (`certs/kissing-wave.json`). Times are wall-clock: the laptop's eight cores for the BLAS rows (the dimension-45 run includes a pause), one NVIDIA L40S for the fused kernel. In all, 9,752,282,639,769,360 pairs and 6,716,665,650,099 contacts; no pair closer than sixty degrees.

The verdict. CERTIFIED in all 19 dimensions (Table 6, Figure 2): $9.75 \times$

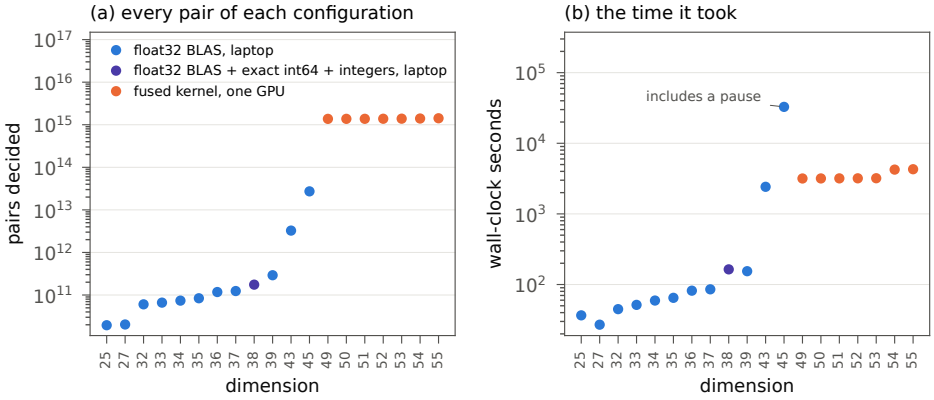


Figure 2: The 19 configurations: (a) the pairs decided in each dimension and (b) the wall-clock time, by the arithmetic used (`certs/kissing-wave.json`). The seven largest ran on one GPU; the dimension-45 point includes a pause.

10^{15} pairs decided, 6.72×10^{12} of them at exactly sixty degrees and none closer; every count matches the published one, and every configuration spans exactly d dimensions. Every bound in the paper holds.

The cost. The 12 laptop rows took 35,954 seconds of wall-clock in all; the 7 GPU rows 6.8 GPU-hours of deciding at $1.09 per hour, and the pod's whole life, its calibration and about 1.3 h idle after it finished included, was billed $8.15 (`corpus/kissing/wave/qiushi-gpu-2026-10-12/pod.json`). The exactness argument is one inequality per path, checked against the data before the run; nothing in the cost is spent on rigour.

What the claimant did next. It published its points, in exactly the form the check needed, and linked the check from both of its READMEs beside its results table. A configuration that holds proves a lower bound and nothing more: it says nothing about optimality, about any upper bound, or about the other steps of the discovery process that produced it. Nor is a bound decided here necessarily the record for its dimension: the field's table of kissing-number bounds (Cohn, 2026) lists higher values in dimensions 25, 27 and 33, from constructions whose points have not been published and which this system has not decided; it lists the bounds decided here in dimensions 32, 34–39, 43 and 45, and does not cover 49–55.

3.2. A curated record: the certificates of an optimization-constants registry

The claim. The optimization-constants registry (Tao et al., 2026) keeps the best known bounds on 115 constants of analysis and combinatorics, each with its source. Many recent entries come from automated discovery and AI-assisted contributors (Georgiev et al., 2025): two of the rows below carry the registry's own note that they were prepared with an AI system, and a

third producer signs its archive "AI Autoresearch" (Numaro, 2026). The registry marks with an asterisk a bound "whose verification is currently minimal", meaning one too large to check by hand and available only as a self-archived artifact. The registry's contributing guide states how the mark comes off: it "is removed when that changes—when the paper is refereed and published, or when the argument is formalized or independently replayed" (Tao et al., 2026). On 2026-09-26 its maintainer, writing that rule into the guide, added of one such bound: "If the 13-point certificate does get independently replayed, or formalized, the asterisk comes off", and that he would take a pull request that removes it on those grounds (Tao, 2026).

Three asterisked lower bounds rest on finite certificates. Two are constants of the sums–differences problem (Katz and Tao, 1999), $C_{3b} \geq 1.77898884$ and $C_{3c} \geq 1.6747338950414058$, each certified by an entropy law, on 13 and 147 points of $\mathbb{Z}^2$ (Mosaic Intelligence, 2026a; Lin, 2026). The third is the Fourier entropy–influence constant (Friedgut and Kalai, 1996), $C_{71} > 6.521845710923046575$, certified by a Boolean function on 18 variables (Numaro, 2026). Beside them, the registry's open pull requests of 2026-10-15 (#210, #217, #221, #225 and #226) offered 10 new bounds: two step functions, five one-line constructions, a nine-dimensional counterexample, a two-law entropy certificate (Jaswal, 2026), and a "certified record package" for the de Bruijn–Newman constant, $\Lambda < 0.1595$ (Mosaic Intelligence, 2026b), against the recorded $0 \leq \Lambda \leq 0.2$ (Rodgers and Tao, 2020; Platt and Trudgian, 2021).

The reading. An entropy certificate is a finite law μ on $\mathbb{Z}^2$ with rational weights. For $(X, Y) \sim \mu$ it claims $C \geq H(X-Y)/\max\{H(X), H(Y), H(X+Y)\}$, with $H(X+2Y)$ added to the maximum for C_{3c}. That is a universal claim over one finite object, decided by the exact pushforward of μ under each linear form and an outward-rounded enclosure of each entropy. The two-law certificate draws 10^{20} independent base-55 digits from two laws, and its entropies add digit by digit only if the map from digits to $L(X, Y)$ is injective. That holds when every form is at most $(55-1)/2 = 27$ in absolute value on both supports, a condition of the claim that is checked, not assumed. The entropy–influence bound is read from a truth table of 2^{18} entries. The integer Walsh–Hadamard transform gives the spectrum exactly, the influence I is a rational, the entropy H of the spectral weights is an enclosure, and the bound $H/(I-1)$ follows from an amplification rule that is cited, not re-proved (O'Donnell and Tan, 2013; Hod, 2017). The record package is read for what it is, a set of scripts and frozen files, and inventoried without running them.

The decision. Each certificate was decided from the bytes the registry cites, every file pinned by digest, without relying on any of the claimants' checkers. The entropy certificates were decided twice, by programs that share no code

with each other or with the claimants: JavaScript over arbitrary-precision integers with dyadic intervals at 128 bits, and the Python standard library alone, its decimal arithmetic at 120 digits rounded outward (Table 5). Both took 1.1 s in JavaScript and 0.2 s in Python, in the rerun kit's timing of 2026-10-02; the queue's two-law certificate was decided within the queue's run below. The truth table was decided in 1.4 seconds by 10 named checks: the digest the artifact binds, balance, Parseval's identity, $I = 261/128$ exactly and above 1, $H > 0$, the bound, the claimant's printed floor inside the enclosure, the record it replaced, and monotonicity. On 2026-10-19 it was decided a second time, in JavaScript with the entropy in closed form, beside 12 planted claims of known truth (Table 5). The queue was decided wherever a finite exact object exists, in 161 seconds for all of it. The record package was read file by file. Of its 104 files, 63 are frozen tables and 5 are scripts, and the reading found that none of the scripts evaluates the zeta function, the heat-flow function H_t or the Dirichlet polynomial whose zeros the bound is about. The analytic inputs enter as frozen outputs whose proofs cite 9 numbered banks that the bundle does not contain.

bound	producer	object decided	verdict here	the registry afterwards
$C_{3b} \geq 1.77898884$	Mosaic Intelligence	13-point law	CERTIFIED, two programs	asterisk removed; [T2026] cited
$C_{3c} \geq 1.6747338950414058$	Y. Lin	147-point law	CERTIFIED, two programs	asterisk removed; [T2026] cited
$C_{71} > 6.521845710923046575$	Numaro	2^{18}-entry truth table	CERTIFIED, one program	asterisk removed; [T2026] cited
$C_{3c} \geq 1.674894211307116$	M. Jaswal	51 and 47-point laws	CERTIFIED, two programs	merged with asterisk, then removed on the replay (#230); [T2026b] cited
$\Lambda < 0.1595$ (the registry's C_{21})	Mosaic Intelligence	104-file package	REFUSED: the evidence is not in it	merged with asterisk, as are all 5 rows after 2021

Table 7: Five bounds of the optimization-constants registry, from the certificate to the registry's record (`certs/sumdiff-ledger.json`, `certs/fei-ledger.json`, `certs/optq-ledger.json`; the registry's bytes after each round of merges in `corpus/optconst-queue/registry-2026-10-18/` and `corpus/optconst-queue/registry-2026-10-18b/`). The queue's other seven rows are CERTIFIED and are not tabled. [T2026] and [T2026b] are the references by which the pages cite the replays.

The verdict. The three asterisked bounds are CERTIFIED, their ratios enclosed as 1.77898884142069354980…, 1.67473389504140587006… and 6.52184571092304657565… (Table 7). Of the queue's 10 rows, 9 are CERTIFIED. For the two-law certificate both implementations enclose the ratio as 1.67489421130711622876434373706 0…; the enclosures share 31 digits, the width of the JavaScript one. One row is REFUSED, with the scope "the evidence is not in the submission": the package certifies that its parts fit together as declared, and whether the parts are true is not in it. That is a verdict on the evidence, not on Λ. The refusal names what would convert it: the bank that holds the floor on the torus, the separator at the barrier point, and the constants of the heat-flow approximation, each a check that would be run once published. One finding is recorded beside the verdicts and is not one of them: the 147-point certificate's own checker encloses its ratio at a hundred digits and then compares in double precision, so it printed OK on all 4 claims it was given, of which 2 are refuted here and 1 is not certified here. The bound the registry prints is true, and it was decided without that checker.

The cost. Seconds per certificate on the laptop, as above; nothing was rented. The inventory of the package is a reading, and the record does not time it.

What the registry did next. The replay of the three bounds was offered as a pull request that quoted the rule back (pull request 216 to Tao et al., 2026), and on 2026-10-18 the maintainer merged it (commit ad40c534). The three asterisks are gone, and each of the 3 constant pages cites the replay as a reference beside the certificate it replays. The same day the registry merged the queue's two-law certificate with its asterisk kept, and the package with its asterisk; it now marks all 5 of that constant's rows after 2021 as not independently replayed. The two-law certificate's replay was then offered in the form the first was accepted (pull request 230), and 3.5 hours later the maintainer merged it (commit 9938edd4). That asterisk is gone too, so the best lower bound the registry lists for C_{3c} without the mark moved from 1.6747338950414058 to 1.674894211307116, and the page cites the second replay, [T2026b], beside the first. The same request restored a replay note on the C_{3c} page, which the merge of the queue's certificate had reverted because its branch was cut before the first merge. In the same minute the maintainer merged the queue's two step functions for C_{1a} and C_{1b}, which were decided CERTIFIED here and reported in a comment on their pull request, and in a commit of his own, pushed with the merge, marked them and the two leaderboard rows beside them with the asterisk, as "replayed only by the submitting team's own checker" (commit 3010f7ed). The two pull requests that cited the rule removed every mark they asked about, and the decision reported only as a comment removed none; the maintainer's

stated reason does not mention the comment, and three cases support no causal conclusion. Those four rows were then decided by two implementations, all four CERTIFIED, and offered in the same form on 2026-10-18 (pull request 232). The registry's newest asterisked row, the trace-constant bound $C_{86} \leq 1.8117251$ merged with its mark the day before, was decided the same way on 2026-10-19: both implementations CERTIFIED, on the six finite conditions of the claimant's criterion, the step measure's density proved positive on 3,658 boxes in each; the chain from those conditions to the bound is the claimant's published one, read and not re-derived. The author of the one-line bounds answered a request for the derivations two of them rest on, and both were decided CERTIFIED from the written derivations, with no code of his run. None of this is peer review: the registry's mark records the verification available, the replay is the verification its rule asked for, and the maintainer applied the rule to it without re-deriving it.

3.3. A formal proof: OpenAI's mathematics release, read against its claims

The claim. In September 2026 OpenAI announced a proof of finite-time blowup for the forced three-dimensional Navier–Stokes equations, one of the four alternatives of the Clay problem. It came with a Lean repository of 2,486 files and 618,762 lines—a Navier–Stokes library, an Euler library and the challenge statements—proved against a statement that DeepMind's Formal Conjectures project had written months earlier (OpenAI, 2026b; Google DeepMind, 2026). The same release carries 719 machine-written manuscripts in 372 families, with 416 Comparator challenges, Lean statements that a proof must satisfy, as its formal spine (OpenAI, 2026a). The release describes itself as "Partial progress", with review "unchecked" and 300 of its 719 manuscripts (42%) formalized at the top line; the claim read here, for both, is that what is formalized has been machine-checked.

The reading. A formal certificate makes two claims (Section 2.4): that the kernel accepts the proof, which a build decides, and that the statement proved is the theorem claimed, which no build decides. The second is read. Clay's wording (Fefferman, 2000), the paper's Theorem 1.1 and the formal statement's hypotheses and conclusions were set side by side, and every difference was recorded. The reading issues no verdict of its own, so a weak point in a 166-page manuscript is recorded as a note, not a refutation. Each challenge of the release makes the same two claims: the kernel's, decided by replaying it, and the statement's, read against the theorem its manuscript claims. A manuscript whose headline rests on a finite object is a claim about that object, decided exactly wherever the object is published.

The decision. 24 files were pinned on 2026-09-09. The repository was built at the announced commit 8937a8f4, over 2,486 modules of its own (11,251 with the library it imports, the other 8,765 taken from the pinned build

cache), in 140 minutes of build time over two legs of 8 and 4 threads (3 h 17 min of wall-clock), and the kernel reported only the standard axioms. Comparator, the claimant's judging tool, accepted both challenge theorems, and a non-vacuity file of 7 declarations compiled against the result. The reading found that the formal statement is Clay's alternatives (C) and (D), with nothing added to the competitor class and nothing dropped from the data conditions. It re-derived 6 steps of the manuscript, recorded 5 slips, none affecting a conclusion, and named 13 weak points. It also found one clause of the paper's Theorem 1.1, the uniform energy bound $\sup_{0\le t<1}\|u(t)\|_{L^2} < \infty$, missing from the formal conclusions at that commit.

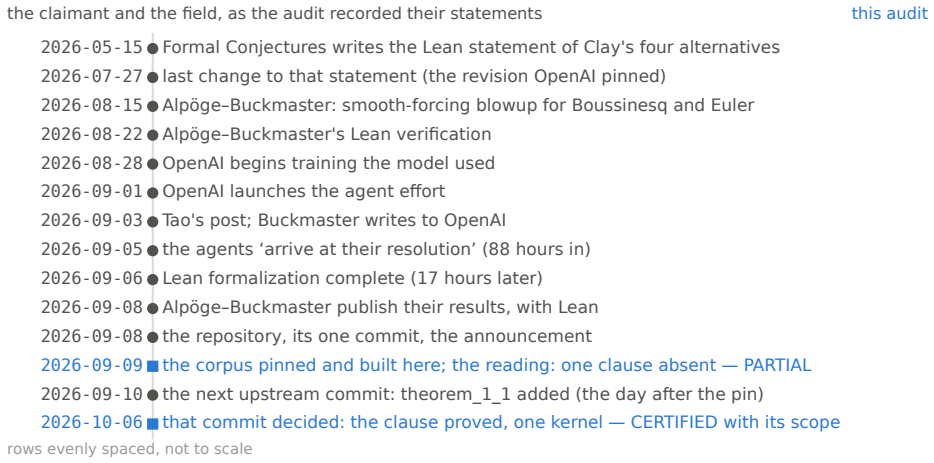


Figure 3: The Navier–Stokes claim in time: the claimant's and the field's dates as the audit recorded them from their statements (`corpus/navier-stokes/audit.json`), the next upstream commit, and the audit's own two rows, in blue.

The verdict. PARTIAL at 8937a8f4 (Figure 3): the kernels' claim, that the proof is accepted, is CERTIFIED by two kernels, and the formal statement proves Clay's alternatives (C) and (D); the energy clause of the paper's Theorem 1.1 is not among its conclusions, and the scope sentence of the record names it. The next upstream commit, f9e8bc5b, was made on 2026-09-10, the day after the audit pinned the repository (188 files changed, 25,143 lines added). It was decided on 2026-10-06 with one kernel: an incremental rebuild of the 486 changed or dependent modules under the pinned toolchain, in 25 minutes, the other 2,173 kept from the pin's build, checked by Lean's kernel only, with the standard axioms; Comparator, the exporter the second kernel reads and Lake all abort at start on this machine's operating system at that toolchain, so neither Comparator nor the second kernel was re-run; the challenge files are byte-identical to the pin's. The clause is now a conclusion of `theorem_1_1`, whose statement is OpenAI's

own, read here and not judged by Comparator, and which the release's own list of main results does not declare. The verdict is CERTIFIED with that scope, the PARTIAL kept as its history (Toledo, 2026a, the Navier–Stokes report).

The release around it. Every challenge was replayed under Lean's kernel and under a second kernel written independently of it (ammkrn, 2026): 267 accepted by the stock second kernel, 127 by it with a 1 GiB stack, and 4 by it with a patch that changes two reduction rules to do them as Lean's reference kernel does; Comparator's sandbox guard was not applied. 398 were accepted by both kernels, 18 were REFUSED for resources (a stack, memory or time limit, on hosted runners and then on rented machines), and none was rejected. Every formal statement was read against the theorem its manuscript claims. Of 242, 127 match; 34 state less than, or other than, what they claim (11 different, 23 narrower without saying so); 38 are narrower and say so; 39 are narrower and not linked to a claim; and 4 are support only. Of 66 manuscripts whose headline rests on a finite object, 44 were decided exactly here, 7 were REFUSED, and 15 assert a witness the release does not publish. Table 8 ranks the 14 gaps between what the release claims and what it checks, found on 2026-10-10 at commit fd4aeeb2 and pinned to file and line before any fix; 6 are headline theorems that no checked statement carries. Nothing in the release was found false.

The cost. The build, 140 minutes; the challenge replays, hosted runners and then rented machines within a pre-registered budget cap. The readings—the Navier–Stokes manuscript in five section memos and a map of its Lean development, and the formal statements against the theorems their manuscripts claim—were produced by AI agents from a written brief, every statement reported as short of its claim read a second time before it was stated. The second readings changed the word of 40 of the 77 statements they covered, and the less severe word was recorded, so that 62 statements narrower without saying so on the first pass became 23. No specialist in the fields concerned has confirmed the readings, and the paper claims them as readings, not as a method.

What happened upstream. The commit of 2026-09-10 proves the clause; it was made the day after the audit pinned the repository, and this paper draws no causal link between the two. An independent paper on the same release argues the general point—that Lean verification of an autoformalized statement does not by itself establish the natural-language theorem (Bastounis et al., 2026); the reading here is the finite instance of it, with the clause named. Each gap of Table 8 is pinned to the file and line it concerns, with the ask that would close it (Toledo, 2026a, the openai/math report).

severity	the gap
1	Hilbert's sixteenth problem: the uniform bound is in no Lean statement
1	Kaplansky: the headline's group is torsion-free; the Lean group is required to have torsion
1	Penrose inequalities: no Penrose inequality is stated
1	Coarse assembly: the headline is about the maximal map; the Lean is about the reduced one
1	Projected Lax: the formalized cone has an exact lift, by the family's own third manuscript
1	Across the release: 34 of 242 formalized families state less than, or other than, what they claim
2	63 displayed definitions Comparator never compares — one of them carries 'the plane is not 5-colorable'
2	Scope notes credit the formalization with theorems no challenge states
2	The independent kernel was switched on for 2 of 416 challenges — and cannot finish the largest as shipped
3	15 headline witnesses are asserted but not published
3	Papers cite checkers and data the release does not ship
3	Shipped checkers that stop short of what their paper claims — each step re-decided here
3	Printed numbers the text alone does not determine
4	Scope notes that are stale or contradict their own challenges

Table 8: What the release claims beyond what it checks (`corpus/openai-math/findings.json`), ranked by the severity the audit assigned, from 1 (a headline theorem that no checked statement carries) to 4; the audit's readings were made by AI agents (Section 5.5). Each row is pinned to the file and line at commit fd4aeeb2 and carries the ask that would close it (Toledo, 2026a, the openai/math report).

3.4. The record beyond the three exercises

Beside the exercises, the public register holds 132 claims from 59 claimants, decided from 25 records between 2026-08-25 and 2026-10-05: 107 CERTIFIED, 2 REFUTED, 2 needing data the claimant has not published, and the rest composite or queued (Toledo, 2026d); of the exercises it holds the Navier–Stokes row and the registry's three asterisked bounds. The same machinery decides engineering numbers over declared envelopes (Toledo, 2026c) and the likelihood fits and contour counts of metocean statistics (Toledo, 2026b), and it grades the rules by which benchmarks grade: of 4,000 wrong answers minted to fall just outside certified enclosures, absolute-tolerance rules accept 3,582, with tolerances from 10^{-3} to 10^{-15} pooled (from 44.4% to 99.4% at a single tolerance), and a rule that tests membership of the enclosure accepts none (Toledo, 2026a, the grader measurement). No production grader is measured on real model outputs. The laboratory's own theorems are decided by the same rule; one of them,

the value of $\lambda(4)$ (Toledo, 2026f), has been re-derived by an outside party with code of its own (rainrzk, 2026), the only such rerun the record holds. The companion papers report each of these in full, and they are not re-argued here.

4. The method evaluated

The exercises show what the method finds. This section measures the method itself, as a decision procedure, on the questions a checker must answer before it is trusted: does it accept what is false, decide what is malformed, or refuse what is true; how does its arithmetic compare with an independent library; how often does it fail to decide what a stronger method could; and what does it cost. A checker that refuses everything never accepts a false claim, so claims known to be true are planted beside the false ones, and a refusal of a true claim counts against the method.

The arithmetic against an independent library. The interval kernel was run on 946,146 seeded cases across 19 operations—up to 50,000 each, drawn from full-precision doubles, subnormals, arguments near overflow and underflow, and boxes that straddle a function's critical points—and every enclosure it returned was compared with the exact result, or for a box the exact range, computed in Arb (Johansson, 2017) at 256 bits and more (Table 9). Every one of the 944,642 enclosures it returned contained the exact value, and it refused the other 1,504; none of the 12 malformed inputs (a NaN, an inverted interval, division through zero, a non-integer or overflowing power, a logarithm at or below zero) was decided; and the comparison caught two planted wrong kernels, a round-to-nearest addition in 20,022 of 50,000 sums and the platform's exponential taken as an enclosure in 50,000 of 50,000 cases, so it can fail. The 1,504 refusals inside the domain are exponentials near underflow, which the kernel refuses rather than enclose. The audit ran in 55 seconds and runs in reduced form in every build. It is a test of the primitives on sampled inputs against code written by other people, not a proof that every path computes an interval extension, and it does not replace a review of the code by a specialist. It covers the double-precision interval kernel and the special functions of the likelihood certifier; the exercises' entropy and entropy–influence deciders, which do not use the kernel, are compared below.

The entropy deciders' arithmetic against Arb. The deciders of Section 3.2 compute in dyadic intervals at 128 bits and more and in Python's `decimal` at 120 and 60 digits rounded outward. Their real functions—the outward conversions, the logarithms, the entropies and ratios, the directed-rounding contexts—were run on 298,746 cases from one seed, among them the registry's own certificates and the enclosures the records hold, and every enclosure was compared with the exact value computed in Arb at

operation	cases	contained	violations	refused	median ulps
$x + y$	50,000	50,000	0	0	2
$x - y$	50,000	50,000	0	0	2
$x \times y$	50,000	50,000	0	0	2
x/y	48,616	48,616	0	0	2
x^2	50,000	50,000	0	0	2
$\lvert x \rvert$	50,000	50,000	0	0	0
x^k	50,000	50,000	0	0	9
exp	48,275	46,771	0	1,504	123
log	49,836	49,836	0	0	5
sin	50,000	50,000	0	0	360
cos	50,000	50,000	0	0	347
tanh	50,000	50,000	0	0	0.5
the platform's cos, padded	50,000	50,000	0	0	4,716
the platform's sin, padded	50,000	50,000	0	0	4,724
$\ln \Gamma$	50,000	50,000	0	0	46
ψ	50,000	50,000	0	0	36
ψ'	50,000	50,000	0	0	22
$P(a, z)$	50,000	50,000	0	0	7,025
Φ	49,419	49,419	0	0	115

Table 9: The interval kernel against Arb (`certs/kernel-arb-audit.json`): for each operation, the cases run, the enclosures that contain the exact value computed in Arb, the violations, the in-domain refusals, and the median width of a point input's enclosure in units in the last place of the larger end of the result (the record holds the maxima too; a median below one ulp, as for tanh, comes from results just below 1). The widths of the padded platform functions are those of the older helper that Section 5.5 names.

256 bits and more, or exactly in rationals when the value is rational (`certs/entropy-arb-audit.json`). Of the 264,031 arithmetic cases, 264,015 returned an enclosure and every one contained the value; the other 16 were refused, each outside the amplification rule's domain. All 71 malformed certificates were refused, and all 7 planted wrong functions were caught. The first run, on 2026-10-19, was not clean, and the record at that commit holds what it found before anything was fixed: 7 kinds of printed enclosure cut toward zero—the strings the entropy–influence decider returned, the lines the Python verifiers print, and the records that copied them, each with its upper end below the value it bounded—and 10 malformed inputs that a Python door decided: a weight of 1.9 read as 1, a Boolean weight, a non-integer coordinate, a one-point law refuted as 0/0, a certificate of another problem, a logarithm at zero. No published verdict was affected, because every verdict compared the full-precision ends; each finding was fixed the same day, and the record keeps all 17 with their witnesses and the fix each received. The audit ran in 112 seconds and runs in reduced form in every build, with the same limits as the kernel's.

Planted claims of known truth. 23 claims whose truth is known by construction were decided by the system's own deciders (Table 10). On the kissing

configurations of E_8 and D_4: a duplicated row, the last row replaced by twice another row (parallel to it), the sum of two contact vectors (at thirty degrees from each), and the zero vector, each false; the configuration negated and its coordinates permuted, each true; rows of different dimension, malformed. On the registry's three entropy certificates: the bound claimed just above the ratio, false; the law translated by an integer vector, which moves no entropy, true; a weight moved off a probability law, and a base too small for the digit map to be injective, malformed. Every false claim was refuted, every true one certified and every malformed one refused, in under a second. The set is small and targeted: it tests the deciders on the objects of the exercises, and it is not an estimate of a rate.

objects	false claims refuted	true claims certified	malformed refused
kissing (E_8, D_4)	8 of 8	4 of 4	2 of 2
entropy certificates (the registry's three)	3 of 3	3 of 3	3 of 3

Table 10: Planted claims of known truth, decided by the system's deciders (`certs/method-eval.json`).

Where a conventional check fails on the same objects. The 147-point certificate's own checker compares in double precision what it encloses at a hundred digits, and printed OK on all 4 of the claims it was given, of which 2 are refuted here (Section 3.2). A bound can also be printed on the wrong side of its own certificate. The registry's $C_{1d} \leq 1.463071582607559$ rests on a 300-cell step function whose exact ratio, the claimant's own fraction, is $1.46307158260755932746\ldots$, above the printed bound: rounded to nearest, as a floating-point print rounds, an upper bound can fall below the value it bounds. The certificate shows $C_{1d} \leq 1.4630715826075594$, and the record holds both verdicts (certs/autocorr-replay.json). Tolerance rules accept most of 4,000 wrong answers minted to fall just outside the certified enclosures (Section 3.4). The kissing release did not check its pairs at all, and the record package's scripts check the inequalities they were written to check and not the zeros the bound is about. In each case the conventional check is the producer's own, and it stops where the claim exceeds it.

How often a reading catches a defect. A reading issues no verdict, but its misses can be counted. From the 30 families of the mathematics release read as MATCHES whose one challenge is at most seventy lines with no definition hole, 24 were drawn with a seed, and in 12 of them one line of the statement was changed so that it states less than the headline, or something else—a constant loosened, a domain restricted, a hypothesis added, a conclusion weakened, a sign reversed—with the scope note silent on it; the

other 12 are the release's bytes. Fresh AI readers, not told that anything was planted, read every packet blind under the pre-registered brief, and, as in the audit, a second reader read every family the first did not read as MATCHES or declared. In both of the 2 pipelines all 12 plants were flagged, each at the planted line, 11 with the key's word (the twelfth was recorded as declared, the case the key marks arguable), and no control was flagged (`certs/reading-recall.json`; every tool call the readers made is kept, and none named a file outside the packets). Twelve of twelve bounds the recall below by 0.735 at 95%, and only for short statements read under a brief that, unlike the audit's, asked for every constant, bound and domain to be checked. The audit's long statements are not measured, and the readers, the author of the plants and the audit's own readers are agents of one model family, whose misses may be correlated.

What the method refuses that a stronger method could decide. Incompleteness has a size, and it is reported. Of the 416 Comparator challenges, 18 are refused for a stack, memory or time limit that more resources might clear. Of the cosine-minimum decisions of the generation loop, 71 are refused because the enclosure straddles the bar, which a finer cover or a higher precision might separate. The kernel's 1,504 refusals near underflow are a design choice: the enclosure $[0, 2^{-1074}]$ would be valid there, and the kernel does not return it. None of these is converted by lowering the rigour; each stays refused with the reason that stopped it, until a stronger method decides it.

Red controls and cost. The control build runs 147 batteries, 91 of which state their red controls, 640 in all; a battery whose red controls do not all fire fails. A red control shows that one named failure is caught, not that the code is free of others, which is why the comparison with Arb above is made at all. Table 11 gives what each exercise and each evaluation cost.

decision	what was computed	cost
kissing, 12 configurations	every pair, on the laptop	35,954 s wall-clock
kissing, 7 configurations	every pair, on one NVIDIA L40S	6.8 GPU-hours, $8.15
the registry's entropy certificates	two implementations	1.1 s and 0.2 s
the registry's truth table	two implementations, 10 checks; the second with its 12 planted claims	1.4 s and 21 s
the trace-constant certificate	two implementations, the density proved on 3,658 boxes each	255 s
the registry's pull-request queue	10 rows	161 s
the Navier–Stokes repository	the build at the pinned toolchain	140 min
the kernel against Arb	946,146 cases	55 s
the entropy deciders against Arb	298,746 cases	112 s
planted claims	23 claims	under a second

Table 11: What the decisions cost, each read from the record that holds it. The readings are not timed and are not a method (Section 5.5).

5. Discussion

5.1. What the three exercises show

The defects are the verifier's kind. No bound was found false and no theorem was false. In each exercise the gap is between what was claimed and what was checked: a pairwise condition proved by structure and never checked pair by pair; a certificate's checker that compares in double precision what it encloses at a hundred digits; a package that certifies its own assembly and not the analysis it assembles; a clause of a theorem that the formal statement leaves out, and formal statements narrower than the theorems beside them. Each was closed, or named, by a finite computation or a recorded reading whose only hard part was deciding to make it.

The producer's checker shares the producer's blind spots. In every case the claimant had a checker, and in every case it stopped where the claim exceeded it. An independent decision is not a better checker; it is a checker that was not written by the claimant and so does not know where to stop.

Refusal is a result, and it has a size. Seventeen kissing rows were NEEDS DATA from the request of 2026-10-06 until the points were released on 2026-10-11 and 2026-10-12, each naming the files that would decide it; the request was answered and the rows closed, an opacity measured and then removed. The record package is REFUSED with the three inputs that would decide it named. The Navier–Stokes reading issues no verdict and defers to the kernel,

so the missing clause became a scope sentence rather than an accusation, and a commit made the next day proves it. Every refusal says what would convert it, and the kinds are never summed (Section 2.2). The cost of exactness, by contrast, is small (Table 11): the objects are integers, rationals and bounds.

A decision changes the record. Each exercise ends outside the system. The kissing claimant published its points in the form the check needed and linked the decision from its release. The registry's maintainer, who had written down beforehand what removes the mark of an unverified bound, removed it from four bounds on the replays and cites them on their pages. OpenAI's next commit, made the day after the audit pinned the repository, proves the clause the reading found missing. We draw no causal conclusion from these cases. They show what the grammar makes possible: a CERTIFIED with its certificate can be cited, a REFUSED that names its files can be answered, and a scope sentence can be closed.

5.2. Independence, counted

"Independent" carries different weights, and they are not interchangeable. Table 12 separates five levels of evidence and places every result of this paper on them. Every result reaches the first level; several reach the second; two reach the third, each for one part of a claim only, the kernels' claim and the interval kernel's arithmetic; none reaches the fourth or the fifth. The $\lambda(4)$ audit, by an outside party, is a third such case, outside this paper. The registry's merge of Section 3.2 is not on the table at all: its maintainer applied his rule to the replay and did not re-derive it, so it is an outcome, not a check.

5.3. Relation to existing practice

Validated numerics and computer-assisted proof are the certifiers' ancestry: interval arithmetic (Moore et al., 2009; IEEE, 2018), interval methods for nonlinear systems (Neumaier, 1990), verification methods in floating point (Rump, 2010, 1999), correctly rounded and ball arithmetic (Fousse et al., 2007; Johansson, 2017), rigorous integration (Kapela et al., 2021) and the proofs they made possible (Tucker, 2002; Hales et al., 2017). Formal proof checks a proof against a stated theorem (de Moura and Ullrich, 2021; The mathlib Community, 2020), and a translation that compiles may still not be the theorem intended (Bastounis et al., 2026). Certifying algorithms return a certificate that a small independent checker verifies (McConnell et al., 2011), as SAT solvers do with proofs that DRAT-trim checks (Wetzler et al., 2014), and proof-carrying code ships a proof of a safety policy with the program (Necula, 1997). Decision procedures already treat a failure to decide as an outcome: SMT solvers answer *unknown*, with a reason

level	evidence	where it holds in this paper	what it cannot catch
1	the same code, rerun from pinned inputs	every record re-read against its pins at build; the registry's and the evaluation's records re-derived by their batteries; the register as of 2026-10-02 (88 rows) re-derived by the rerun kit, 0 moved; the kissing pairs not re-derived at build	any defect in the code itself
2	a second implementation by the same author, from the same specification	Table 5: the kissing samples, the entropy certificates, the entropy–influence bound, the step functions, the trace-constant certificate, δ_3, three engineering decisions; written by AI agents from one brief	a misreading the two share
3	an implementation by another party	the Lean kernel and a second kernel for the kernels' claim (Section 3.3); Arb for the interval kernel's arithmetic, on sampled inputs (Section 4); outside this paper, the $\lambda(4)$ audit, by an outside party (1 such rerun)	a defect outside the part re-derived
4	review of the specification and the scope by a specialist	none: the readings were made by AI agents and no specialist has confirmed them	—
5	a formal proof of the metatheory	none: Proposition 1 is a paper proof; the exact-class certificates are in a form a proof assistant could check	—

Table 12: Five levels of evidence, and where each result of this paper stands on them. They are counted apart and never summed.

(Barrett et al., 2017); proof platforms record a prover's result as valid, invalid, unknown, timeout or out of memory (Filliâtre and Paskevich, 2013); δ-complete procedures decide nonlinear real arithmetic up to a declared perturbation (Gao et al., 2012, 2013); and nonlinear inequalities over special functions and in the Flyspeck project are proved by interval and polynomial methods inside or beside a proof assistant (Akbarpour and Paulson, 2010; Solovyev and Hales, 2013). Verified checkers close the gap between a certificate and its checker (Tan et al., 2021). Reproducibility initiatives re-run published computations (Rougier et al., 2017), and the archiving of source code and reproducible builds pin the bytes a result came from (Di Cosmo and Zacchiroli, 2017; Lamb and Zacchiroli, 2022). Table 13 sets them side by side on the properties this system enforces. The three values are not new: SMT's *unknown* and a proof platform's statuses already record a failure to decide. What this system adds is where it points them and what it keeps: the computations are aimed at other parties' published claims, read from

their bytes; every verdict carries its witness and a scope sentence; and the inputs and code that decided each claim are pinned and re-derived. Each property exists elsewhere; this system enforces them together, by code that refuses. Its own weakest point is the last column: its semantics has a paper proof and its implementation is tested, not formally verified.

5.4. Threats to validity

- *Selection.* Every claim was chosen by the author: the register records every row as self-initiated, and the three exercises were picked because they had finite objects to decide. The results describe these claims, not machine-generated mathematics at large.

- *The author in the outcomes.* The outcomes of Section 3—the points published, the marks removed—followed requests and pull requests the author made. They are reported as what happened, and no causal claim rests on them.

- *Correlated implementations.* The deciders and their second implementations were written by AI agents under the author's direction, from shared briefs. N-version programs written that way can fail together (Knight and Leveson, 1986), and level 2 of Table 12 is counted apart for that reason.

- *A small evaluation.* The 23 planted claims, and the 12 on the entropy–influence certificate, are targeted, not sampled, and give no rate; the comparisons with Arb cover the double-precision kernel and the entropy deciders' arithmetic on sampled inputs, not every input.

- *The readings.* The statement readings and the Navier–Stokes reading were made by AI agents and checked only by second readings made the same way, which changed the word of 40 of 77 statements. Their recall was measured once, on 12 defects planted in short statements and read under an alert brief (Section 4); on the audit's long statements it is not measured.

- *No real refutation in the exercises.* No claim of the three exercises was refuted. The REFUTED verdicts here are planted ones and the printed C_{1d} bound of Section 4, refuted as printed while its certificate shows a slightly weaker bound.

- *The grader measurement.* The wrong answers were minted to fall just outside the certified enclosures and were scored by generic tolerance rules; no production grader was measured on real model outputs.

approach	decides other parties' claims	refusal is a recorded verdict	evidence pinned and re-derived	checker independent of the producer	guarantee of the checker
validated-numerics libraries (Fousse et al., 2007; Johansson, 2017; Rump, 1999; Kapela et al., 2021)	no: computes enclosures; the user decides	no	not enforced	—	tested libraries
computer-assisted proofs (Tucker, 2002; Hales et al., 2017)	one theorem each, the authors' own	no	code published	no	formal, for Flyspeck
proof assistants, independent kernels (de Moura and Ullrich, 2021; ammkrn, 2026)	proofs of stated theorems	accept or reject	a reproducible build	the kernel; not the statement	a small trusted kernel
certifying algorithms, independent checkers (McConnell et al., 2011; Wetzler et al., 2014)	a solver's output, with a certificate	accept or reject	the certificate	yes	a small checker
proof-carrying code (Necula, 1997)	a program's safety policy	accept or reject	the proof travels with the code	yes	formal
SMT solvers, proof platforms (Barrett et al., 2017; Filliâtre and Paskevich, 2013; Gao et al., 2013)	formulas a user poses	yes: unknown, with a reason	not enforced	—	the solver, tested; verified checkers for some
reproducibility initiatives (Rougier et al., 2017)	a published computation, re-run	reproduced or not	yes	reviewers	none
this system	published claims, from their bytes	yes, with the reason	every input pinned by digest; re-derived at build	shares no code with the claimant	paper proof of the semantics; implementation tested

Table 13: This system beside existing practice, on the properties it enforces. Each property exists elsewhere; the combination, aimed at other parties' published claims and enforced by code that refuses, is what is claimed.

5.5. What is not claimed, and the limits

Not claimed. Interval arithmetic, the Krawczyk operator, floating-point filters, sums of squares and Sturm sequences are not claimed; they are cited. The validity of a declared model is not claimed: a PROVED engineering number holds for the declared model over the declared envelope, not for the sea or the reservoir. A kissing bound "decided here" is a statement about one published configuration, not about the record in that dimension. Nothing is certified in the regulatory sense; the word is used in the mathematical sense of Definition 3 only. Nothing is a probability.

Locality and incompleteness. A certified maximum is the maximum in its box; a global statement over a family would need a branch-and-bound over the family's whole parameter space, which the system does not yet run. The system refuses what it cannot decide, and the refusal rate is a property of each campaign, published with it.

Readings are not decisions. Three results of this paper rest in part on readings rather than computations: the clause found absent from the Navier–Stokes formal conclusions, the statements of the mathematics release found short of their claims (Table 8), and the inventory of the record package of Section 3.2. Each reading was produced by AI agents from a written brief; the second readings were made the same way, so they are a check on consistency and not an independent review; and no human specialist in the fields concerned has confirmed them. The clause is the one upstream later proved, which is consistent with the reading and does not confirm it. Their recall has been measured once, on defects planted in short statements (Section 4), which says nothing about the long ones. Until a specialist does, the gaps are reported as what the release does not show, never as errors.

Soundness rests on code. Proposition 1 is a theorem about interval extensions; whether a program computes one is a fact about the program. The defences—second implementations, red controls, pinned bytes, calibrations, outside reruns (Section 2.5)—reduce that risk and do not remove it. A red control shows that one named failure is caught, not that the code is free of others, and two implementations written from one specification can agree and both be wrong. The defences are also uneven. Some certifiers have a second implementation that shares no code. Some, among them the point-in-polygon decider, the cosine-minimum certifier and the likelihood certifier, are re-run in the browser by the same bytes, which checks the record and not the code.

What a read of the code for this paper found, and what was closed. An adversarial read of the code on 2026-10-09 found defects under the system's one-line description ("independent exact certification of machine-generated mathematics—exact arithmetic, no code shared with the claimant, refusal

as a verdict"), and they were closed the same day, with the correction published beside the old numbers (`certs/erratum-2026-10-09.json` pins the numbers, the commit and the pages that carried them). The most serious: the generation loop's closed-form hunt compared floating-point candidates with certified enclosures and counted a miss as a refutation, so that 54.6 million of its "refutations" were float comparisons. Every candidate is now an exact rational or an enclosure, and a refutation is a proved disjointness. A word that covered both a proved failure and a straddle was split, a straddle now being REFUSED; a comparison of lower end with lower end was replaced by the champion's enclosure; the kernel gained its NaN and overflow refusals; and the build stopped writing pages with a red battery. None of this moved a verdict reported in the companion papers. A later defect of the same kind was found by a second implementation: the entropy–influence decider of Section 3.2 had two words, so a broken binding, a contradicted influence and a bar inside its enclosure were all printed REFUTED; of the 12 planted claims it printed 6 refusals as refutations, one of them a true claim, and its record printed both ends of the enclosure as truncations, the upper end below the value. Both were corrected on 2026-10-19; its one published row was CERTIFIED, which two words get right, and it is unchanged. The comparison of the entropy deciders' arithmetic with Arb (Section 4) found, on its first run, 17 more of the same kind—printed enclosures cut toward zero, doors that decided malformed certificates—none of which moved a verdict; they were fixed the same day, and the record keeps them. Open, and named in the repository's debt register:

- the older helper that encloses the platform's cosine by a derived pad, which contained the exact value in every case of Table 9 but is wide, is still called from six files, among them the radii-polynomial certifier, whose records are re-run when it is retired;
- one quadrature routine, rigorous on dyadic cells of $[0, 1]$, is called on another interval by one instrument;
- one battery labels some positive assertions as red controls;
- in the engineering demonstrator a PROVED consistency check means that the claim meets an outer enclosure, and the discharge thresholds are rounded outward to a tenth of a bar in floating point rather than in directed rounding;
- the transcendental tails are safe by a margin of twenty orders rather than by directed rounding;
- the same-author second implementations are not independent re-derivations (Table 12);

- only the exact-class certificates are in a form a program other than this system's can check.

The system's own rule applies to each: a check that has not been made is not a check.

6. Conclusions and recommendations

cert-machine decides the numbers and claims that machines propose with three verdicts, each carrying the object that justifies it; the interval and exact verdicts are sound under the inclusion property, which the arithmetic is built to secure, and every other pathway states what it rests on. Its certifiers reduce the questions that arrive—a zero, a maximum, a statement for every input, a sign, a nonnegative polynomial, a minimum, a formal proof—to finitely many exact or outward-rounded decisions; its discipline keeps them honest by code that refuses; and its record is public and reruns. The three exercises are what the method finds when it is pointed at machine-generated claims of three kinds: every one of the 19 configurations holds, pair by pair; the certificates behind a registry's unverified bounds hold, and the registry removed the marks on the replay, while a package that certifies only its own assembly is refused with the inputs that would decide it; and a formal proof is a proof of what it states and not of the clause it omits, until the clause is proved. Beside them, tolerance rules accept most wrong answers minted to fall just outside certified enclosures. Measured as a decision procedure, the system refuted every planted false claim, certified every planted true one and decided no malformed input, and its double-precision interval kernel, checked against an independent library, contained the exact value in all 944,642 enclosures it returned and refused the other 1,504; what it could not decide, it refused, with the reason. What the system offers those who produce such claims is not a second opinion but a different object beside their number: a verdict with its witness, in which a refusal says what would decide it.

Recommendations. To producers of machine-generated mathematics: publish the object, not only its description—the points of a configuration in one integer form, the witness a theorem rests on, the analytic inputs a certificate freezes, the data a fit was made from—with a digest beside each file, because the finite check is cheap once the bytes exist and impossible until they do; state the scope of every check in the release itself ("pairwise inner products checked: false" is the right sentence, and it was the one that made the exercise of Section 3.1 possible); and when a formal statement is narrower than the theorem it accompanies, say so where the theorem is stated. To those who maintain the tables and records of a field: distinguish a bound that has been decided from outside from one that has only been claimed, write

down what removes the distinction—the registry of Section 3.2 does it with one mark and one sentence, and that sentence is what let an outside replay change its record—and ask for the points. To those who build benchmarks and graders: where a reference answer has a certified enclosure, grade by the enclosure and not by a window, because the window's false positives can be minted without limit; where it has none, say which digits of the key are known; and plant red controls, because a grader that has never rejected a planted wrong answer has not been tested—the same holds for a certificate's own checker, which should compare at the precision it encloses. To those who verify: refuse where you cannot decide, name the reason, and count the kinds of refusal apart—a result that says what would decide it is worth more than a probability that nothing could.

Declaration of competing interest

The author declares no competing financial or personal interests. The Qiushi Engine group, OpenAI, the maintainer and contributors of the optimization-constants registry, and the maintainers of the benchmarks and tables named here have not been consulted on this draft; the exchanges with the Qiushi Engine group and with the registry took place on the public threads cited, the Qiushi READMEs link the checks of Section 3.1, and the registry's pages cite the replay of Section 3.2. Prospective co-authors are reviewing this manuscript.

CRediT authorship contribution statement

Carlos Toledo: conceptualization, methodology, software, validation, investigation, data curation, writing—original draft, writing—review and editing. The author list and the roles are to be agreed with the prospective co-authors.

Data availability

Every record, certifier, figure source and table source is public in the cert-machine repository (`https://github.com/carlostoledo1891/cert-machine`, commit 7c3b736d21ec; archived as Toledo, 2026a): the register `certs/claims-ledger.json`; the kissing configurations' decisions `certs/kissing-wave.json` and the GPU pod's results and cost `corpus/kissing/wave/qiushi-gpu-2026-10-12/`; the registry's replays `certs/sumdiff-ledger.json` and `certs/fei-ledger.json`, its queue `certs/optq-ledger.json`, and its pages after each round of merges `corpus/optconst-queue/registry-2026-10-18/` and `corpus/optconst-queue/registry-2026-10-18b/`; the Navier–Stokes audit `corpus/navier-stokes/` (the manifest of pinned files, the

build, the reading, the upstream decision); the openai/math audit `corpus/openai-math/` and `certs/openai-math-*.json`; the grader measurement `certs/envs-record.json`; the registry's step functions `certs/stepfun-replay.json` and `certs/autocorr-replay.json`; the kernel against Arb `certs/kernel-arb-audit.json` and the planted claims `certs/method-eval.json`; the batteries, the rerun kit and the outside reruns under `certs/` and `corpus/`. The macros of this manuscript are written by `tools/paper-numbers/cert-machine-method.js` and by the generators of the companion papers from those records, and a record that no longer supports a sentence makes the build refuse. The claimants' data are cited at their sources and pinned by digest in the records; the Qiushi Engine's point sets are CC BY 4.0, and the OpenAI repositories carry their own licences.

Declaration of generative AI and AI-assisted technologies

During the preparation of this work the author used large language models (Claude, Anthropic) to draft code, exposition and this manuscript in a certified-verification setting: constructions and prose were proposed with the help of the models, and every quantitative claim is decided by outward-rounded interval arithmetic or exact rational arithmetic, with adversarial controls that must fire for a record to be accepted. After using the tool the author reviewed and edited the content and takes full responsibility for the content of the publication. The records, not the models, are the evidence offered; they are public and re-runnable. For this paper specifically: the deciders, the certifiers' code and the readings of Sections 3.2 and 3.3—the section memos and Lean map of the Navier–Stokes manuscript, the statement readings of the mathematics release, the inventory of the record package—were produced by AI agents (Claude) under written briefs and consolidated into the public records; no verdict here rests on a reading alone, and no human specialist has confirmed the readings (Section 5.5). A referee-style reading of the previous draft by an AI agent was applied in this version.

Funding

This work received no external funding. The rented compute of Section 2.1 ($8.15 for the seven largest kissing configurations; the challenge re-runs within a pre-registered cap) was paid by the author.

Appendix A. Reproduction

Every number in this paper is a macro written from a record of the repository at commit 7c3b736d21ec, and a record that no longer supports a sentence stops the build. Each headline result re-derives with the command

beside it in Table A.1, on the laptop of Section 2.1 unless the row says otherwise; each command that fetches a claimant's file refuses it unless it matches the pinned digest, and each battery fails unless its red controls fire. The references are checked too: `node tools/check-bib.js --verify` resolves every entry at its source and records the answer, and the build refuses an entry with no verified row.

result	command	expected	time
kissing, $d = 25$, 27	`python3 tools/run-kissing-wave.py --only qiushi-25, qiushi-27`	CERTIFIED; the constructions rebuilt from the repository's files	as in Table 6
kissing, $d = 32$–37, 39	`python3 tools/run-kissing-configs.py <d>`	CERTIFIED; the pairs and contacts of Table 6	as in Table 6
kissing, $d = 38$	`python3 tools/run-kissing-config38.py`	CERTIFIED	as in Table 6
kissing, $d = 43$, 45	`python3 tools/run-kissing-sections.py <d>`	CERTIFIED; the Gram form by `tools/run-kissing-gram.py qiushi <d>`	as in Table 6
kissing, $d = 49$–55	`tools/runpod-qiushi-lifting.sh`, on a CUDA machine	the calibration on 32 and 39 first, then CERTIFIED	6.8 GPU-hours
the registry's certificates	`python3 tools/verify_sumdiff.py`; `python3 tools/verify_twolaw.py`	CERTIFIED; their red controls refuted or refused	0.2 s;—
the registry's records	`node tools/run-sumdiff-ledger.js --check`; `python3 tools/run-fei-ledger.py --check`; `python3 -I tools/run-optq-ledger.py`	re-derived identical; the queue 9 CERTIFIED, 1 REFUSED	1.1 s; 1.4 s; 161 s
the registry's response	`node tools/read-optconst-registry.js --check`	re-read identical from its pinned bytes	—
the registry's step functions	`node tools/replay-stepfun.js`; `node tools/replay-autocorr.js`	the four C_{1a}, C_{1b} rows CERTIFIED; C_{1c} CERTIFIED; the printed C_{1d} REFUTED, its repair CERTIFIED	—
Navier–Stokes	the build that `corpus/navier-stokes/build.json` records	both challenge theorems accepted; only the standard axioms	140 min
the release's challenges	`.github/workflows/openai-math-kernel.yml`; `tools/runpod-openai-math.sh`	398 accepted by both kernels, 18 refused for resources, none rejected	hosted runners
the registry's truth table, a second time	`node tools/verify_fei.js`	CERTIFIED in both implementations; the 12 planted claims as expected	21 s
the trace-constant certificate	`node tools/replay-c86.js`	CERTIFIED in both; 6 red controls REFUTED in both	255 s
the kernel against Arb	`node tools/audit-kernel-arb.js`	no violation; every malformed input refused; both planted kernels caught	55 s
the entropy deciders against Arb	`node tools/audit-entropy-arb.js`	no violation; every malformed input refused; every planted function caught	112 s
planted claims	`node tools/run-method-eval.js`	every false claim refuted, every true one certified, every malformed one refused	under a second
the register as of 2026-10-02 (88 rows)	the rerun kit, `RERUN.md`	0 rows moved, 0 failed	22 min

Table A.1: Each headline result, the command that re-derives it, what it must print, and what it took (the times are those the records hold;—where a record holds none).

Appendix B. The arithmetic in detail

Elementary and special functions. The certifiers of this paper trust no library function, because the platform's `Math.exp` and `Math.log` carry no accuracy guarantee in ECMAScript—with one exception that Section 5.5 names: an older helper that encloses the platform's cosine by a derived pad rather than by a series is still called from six files, among them the radii-polynomial certifier of Section 2.4, and its retirement for the series below is scheduled with the re-run of those records. Each function is a truncated series evaluated in interval arithmetic plus an explicit bound on the omitted tail: exp after reduction $x = k \ln 2 + r$, $|r| < 1$, by twenty-four Taylor terms and the tail $|r|^{25}/25!\,(1-|r|/26)^{-1}$; log after $x = m\,2^e$ by the series of atanh in $v = (m-1)/(m+1) \in [0, 1/3)$, whose one-sided tail is valid because $v \geq 0$; sin and cos after reduction by an interval enclosure of $\pi/2$, with the box's enclosure forced to ± 1 wherever it may contain an extremum; $\ln 2$ and π in exact rationals by atanh(1/3) and Machin's formula, converted outward. Monotone functions on wide boxes are evaluated at the endpoints. The special functions follow the same pattern: Γ by Spouge's formula with its error bound taken ten times over; $\ln \Gamma$, ψ and ψ' by Binet's expansion after shifting the argument above twenty, the first omitted term as the bound; the regularized incomplete gamma function by its series with a geometric tail; Bessel functions by their power series with a certified ratio bound; erfc by Laplace's continued fraction, whose consecutive convergents bracket the value (Abramowitz and Stegun, 1964); and inverses (Φ^{-1}, P^{-1}) by bisection that moves an end only on a certified sign. The tails are computed in round-to-nearest and are many orders below the one-ulp widening that absorbs them; that margin, not the tail's own rounding, is what makes them safe, and it is stated where it is used.

Long sums. A running interval sum rounds outward at each of n additions, so its width grows linearly in n; at the 175,320 hourly values of one buoy it is a thousand times what the terms carry. Each endpoint stream is instead summed by the compensated Sum2 algorithm (Dekker, 1971; Ogita et al., 2005) and the error bound of Ogita et al. (2005) is added outward with a factor of two to spare, which is what makes a likelihood over a hundred thousand data an enclosure narrow enough to rank families by (Toledo, 2026b).

Derivatives. Gradients and Hessians that a certifier needs come from second-order forward differentiation over the same interval arithmetic (a value, a gradient and a Hessian carried through every operation), or from univariate Taylor jets, so that the derivative a proof uses is an extension of the function the proof is about. The remaining families carry hand-written scores and Hessians, evaluated in the same interval arithmetic.

Further certifiers. The certifiers of Table 3 that the exercises do not use.

A maximum in a box

Proposition 3. *Let ℓ be twice continuously differentiable on a convex box X, let $\nabla\ell$ have exactly one zero θ^* in X, and let the Hessian of ℓ be negative definite at every point of X. Then θ^* is the unique maximizer of ℓ on X.*

Proof. A function whose Hessian is negative definite on a convex set is strictly concave there, and a critical point of a strictly concave function is its unique maximizer on the set. □

Negative definiteness over the box is proved one of two ways: by Sylvester's criterion on the interval Hessian, its leading principal minors Δ_k enclosed and proved to satisfy $(-1)^k\Delta_k > 0$ over all of X (for up to three parameters, with the two enclosures of each symmetric entry intersected); or, where the box's minors are too wide, at the candidate point together with a proof that every Hessian over X is nonsingular—$|I - AH(X)|\,u < u$ componentwise for a positive vector u and an approximate inverse A—so that, X being connected and the eigenvalues continuous, no Hessian over X can change its inertia. This is how a maximum-likelihood fit becomes a certificate (Toledo, 2026b). The statement is local: that no better maximum exists elsewhere in the family remains the search's claim, and every fit is stated with that scope.

A supremum at a limit

Some families have no interior maximum on some data: the likelihood rises toward a limit member, and an optimizer prints the last iterate as the fit.

Proposition 4. *Let $\ell(\mu, \sigma, Q)$ be continuous on $B \times [0, q_1]$ and differentiable in Q on $B \times (0, q_1]$, and let $\partial\ell/\partial Q < 0$ there. Then for every $(\mu, \sigma) \in B$ and every $Q \in (0, q_1]$, $\ell(\mu, \sigma, Q) < \ell(\mu, \sigma, 0)$.*

The proof is the mean value theorem. With $Q = 1/\sqrt{\alpha}$ the generalized gamma reaches the lognormal at $Q = 0$ (Prentice, 1974), and in these coordinates $Q = 0$ is an ordinary point; a proof that the Q-score is negative over a neighbourhood of the lognormal fit excludes the members near the family's edge *with the proof* that each is less likely than its limit. A threshold on α in place of this proof hands a different family the verdict at 544 of 7,767 cell-blocks of the extreme-value atlas (Toledo, 2026b), which is the cost of a threshold measured.

Monotone implicit functions

Where a model quantity is the root of an equation monotone in its parameters, the root's range over the box is bracketed by the roots at two corners, and each end counts only when the equation's sign just beyond it is proved. Colebrook's friction law is handled this way: with $x = 1/\sqrt{f}$ the root of $h(x) = x + (2/\ln 10)\ln(a + bx)$, increasing in x, a and b, every root over the box lies in $[x^*(a_+, b_+), x^*(a_-, b_-)]$, and a floating-point Newton iteration only locates the ends (Toledo, 2026c).

Multilinear forms

Lemma 1. *Let f be multilinear on a box $X \subset \mathbb{R}^n$ (affine in each variable when the others are fixed). Then $f(X) = [\min_v f(v), \max_v f(v)]$, the minimum and maximum taken over the 2^n vertices of X.*

Proof. Fix all variables but one; f is affine in it, so its extremes over that side are at the side's ends. Induct over the variables to reach a vertex for each extreme. f is continuous and X connected, so every value between is attained. □

An emissions-abatement estimate, a sum of products of premises each declared as an interval, is such a form. The kernel checks multilinearity syntactically—every term a monomial in distinct variables, or the corner rule is not applied—and evaluates the 2^n corners in exact rationals; the enclosure is then the exact range, not an outer bound, and the two corners at its ends are the witnesses of a split refusal.

A sign, exactly

Geometric predicates—which side of an edge a point lies on—are signs of small determinants. Each is evaluated first in double precision and trusted only when its magnitude clears a forward-error bound proved for the inputs (Higham, 2002; Fortune and Van Wyk, 1996): with coordinates bounded by M and $u = 2^{-53}$, the computed 2×2 orientation determinant is within $48uM^2$ of the exact one. Below the bound, the sign is recomputed exactly in integers, each decimal literal scaled to a fixed sixty-four places; a literal that would need more is refused rather than rounded. A point on an edge is reported ON, an answer a floating-point test does not have. This is the static filter of exact geometric computation (Shewchuk, 1997; Yap, 1997), written against the literals a file contains rather than against their doubles. Over a contour benchmark it decided 6.37×10^8 predicates, 237,173 of them in integers (Toledo, 2026b).

A polynomial, nonnegative

A claim $p \geq \gamma$ is certified by an identity $p - \gamma = \sum_i c_i q_i^2$ with rational polynomials q_i and rational weights $c_i \geq 0$, checked by exact coefficient comparison; a negative weight or a nonzero residual refuses, and the residual is printed. Finding the decomposition is a semidefinite program (Parrilo, 2003; Peyrl and Parrilo, 2008) that is never relied on: the solver proposes, the identity decides. A separate refuter evaluates p at an exact witness point and refutes the claim if $p(w) < \gamma$.

A minimum of a cosine polynomial

The minimum over x of $\sum_{a \in A} \cos(ax)$ is the minimum over $y \in [-1, 1]$ of $P_A(y) = \sum_a T_a(y)$, a polynomial with integer Chebyshev coefficients. The certifier isolates the critical points with a Sturm chain of the square-free part of P_A' on a slightly enlarged interval, refines each by interval Newton in rationals with outward balls around exact Horner values, encloses P_A on each critical interval with the bound $\sum_{a \in A} a^2(a^2 - 1)/3$ on $|P_A''|$, from V. Markov's inequality for each T_a, adds the endpoints ± 1 exactly, and returns the hull of the smallest enclosures. A stronger certificate identifies the minimum as an algebraic number—a resultant, a Sturm count of one inside the enclosure, irreducibility modulo a prime—and is CERTIFIED only when all of these hold.

Appendix C. Thresholds

Let the envelope depend on a declared scalar p through nonempty boxes $B(p)$. Two cases occur, and they behave differently.

Definition 4 (thresholds)**.** (a) *Translated envelopes.* When p is an operating choice, the boxes move with p. If the verdict is monotone in p—CERTIFIED at p implies CERTIFIED at every $p' \leq p$, and REFUTED at p implies REFUTED at every $p' \geq p$—the *flip thresholds* are $p_{\mathrm{C}} = \sup\{p : \text{CERTIFIED over } B(p)\}$ and $p_{\mathrm{R}} = \inf\{p : \text{REFUTED over } B(p)\}$. (b) *Nested envelopes.* When p is one end of an interval that a measurement could narrow, oriented so that $B(p') \subseteq B(p)$ whenever $p \leq p'$, the *decision threshold* is $p^* = \inf P$, where P is the set of p at which the claim is CERTIFIED, or REFUTED under the epistemic reading, over $B(p)$.

Proposition 5. *(a) For translated envelopes under monotonicity, if the procedure has certified the claim over $B(\underline{p})$ and refuted it over $B(\overline{p})$, then the claim is* CERTIFIED *for every $p \leq \underline{p}$ and* REFUTED *for every $p \geq \overline{p}$, and $\underline{p} \leq p_{\mathrm{C}} \leq p_{\mathrm{R}} \leq \overline{p}$. (b) For nested envelopes, if every G_j is inclusion isotone ($X \subseteq Y$ implies $G_j(X) \subseteq G_j(Y)$) and the procedure has certified the claim, or refuted it under the epistemic reading, over $B(p_0)$, then it reaches the same verdict over every $B(p)$ with $p \geq p_0$, and $p^* \leq p_0$.*

Proof. (a) The two implications are the monotonicity hypothesis applied at $\underline{p}$ and at $\overline{p}$, which also give $\underline{p} \le p_{\mathrm{C}}$ and $p_{\mathrm{R}} \le \overline{p}$. If $p_{\mathrm{R}} < p_{\mathrm{C}}$, monotonicity would give a p at which the claim is both CERTIFIED and REFUTED; by Proposition 1, φ would then hold at every point of the nonempty $B(p)$ and fail at one of them, a contradiction. (b) Intersect each box of the cover of $B(p_0)$ with $B(p)$. The pieces are boxes that cover $B(p)$, and by isotonicity each enclosure lies inside the one that decided, so the same inequalities hold on every piece; $p^* \le p_0$ is then the definition. □

Monotonicity in case (a) is a property of the model and is proved in each use; where it is not available, the procedure reports the two endpoints it decided and no threshold. In case (b) there is a single threshold: certification and epistemic refutation are both kept when the box shrinks, so they lie on the same side of p^*, and which of them holds there is what the narrowing would reveal. A universal refutation is not kept, because its counterexample may lie outside the smaller box. The natural extensions used here are inclusion isotone, as their primitives are (Moore et al., 2009). The procedure computes the endpoints by bisection over rational p, each endpoint decided like any other claim, or in closed form where p enters additively, the result rounded outward to the precision reported. The engineering demonstrator has one of each (Toledo, 2026c): the pump discharge, an operating choice, in closed form, monotone because the wellhead pressure is the discharge plus a term that does not depend on it; and the lower end of the roughness interval, which an inspection could narrow, by bisection over nested boxes, with one threshold. A threshold turns a refusal into a measurement or an operating change with a known effect on the verdict.

References

Abramowitz, M., Stegun, I.A., 1964. Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables. volume 55 of *Applied Mathematics Series*. National Bureau of Standards, Washington. URL: `https://personal.math.ubc.ca/~cbm/aands/`.

Akbarpour, B., Paulson, L.C., 2010. MetiTarski: An automatic theorem prover for real-valued special functions. Journal of Automated Reasoning 44, 175–205. doi:`10.1007/s10817-009-9149-2`.

ammkrn, 2026. nanoda_lib: a type checker for Lean 4 export files. URL: `https://github.com/ammkrn/nanoda_lib`. Commit 3a240721, with the patch of issue #44.

Barrett, C., Fontaine, P., Tinelli, C., 2017. The SMT-LIB standard: Version 2.6. URL: `https://smt-lib.org/papers/smt-lib-reference-v2.6-r2017-07-18.pdf`. Department of Computer Science, The University of Iowa.

Bastounis, A., Circelli, F., Hansen, A.C., 2026. Navier–Stokes lost in translation: why Lean verification of AI autoformalisation does not guarantee correct natural language proofs. arXiv:2610.08144.

Cohn, H., 2026. Kissing numbers. URL: https://cohn.mit.edu/kissing-numbers. The table of bounds, read 2026-10-11; the extract is pinned in the repository.

Conway, J.H., Sloane, N.J.A., 1999. Sphere Packings, Lattices and Groups. 3rd ed., Springer, New York. doi:10.1007/978-1-4757-6568-7.

Dekker, T.J., 1971. A floating-point technique for extending the available precision. Numerische Mathematik 18, 224–242. doi:10.1007/BF01397083.

DeMillo, R.A., Lipton, R.J., Sayward, F.G., 1978. Hints on test data selection: help for the practicing programmer. Computer 11, 34–41. doi:10.1109/C-M.1978.218136.

Di Cosmo, R., Zacchiroli, S., 2017. Software heritage: Why and how to preserve software source code, in: iPRES 2017: 14th International Conference on Digital Preservation. URL: https://hal.science/hal-01590958.

Fasi, M., Higham, N.J., Mikaitis, M., Pranesh, S., 2021. Numerical behavior of NVIDIA tensor cores. PeerJ Computer Science 7, e330. doi:10.7717/peerj-cs.330.

Fefferman, C.L., 2000. Existence and smoothness of the Navier–Stokes equation. URL: https://www.claymath.org/millennium/navier-stokes-equation/. The Clay Mathematics Institute's official problem description.

Filliâtre, J.C., Paskevich, A., 2013. Why3 — where programs meet provers, in: Programming Languages and Systems (ESOP 2013), pp. 125–128. doi:10.1007/978-3-642-37036-6_8.

Fortune, S., Van Wyk, C.J., 1996. Static analysis yields efficient exact integer arithmetic for computational geometry. ACM Transactions on Graphics 15, 223–248. doi:10.1145/231731.231735.

Fousse, L., Hanrot, G., Lefèvre, V., Pélissier, P., Zimmermann, P., 2007. MPFR: A multiple-precision binary floating-point library with correct rounding. ACM Transactions on Mathematical Software 33, 13. doi:10.1145/1236463.1236468.

Friedgut, E., Kalai, G., 1996. Every monotone graph property has a sharp threshold. Proceedings of the American Mathematical Society 124, 2993–3002. doi:10.1090/S0002-9939-96-03732-X.

Gao, S., Avigad, J., Clarke, E.M., 2012. δ-complete decision procedures for satisfiability over the reals, in: Automated Reasoning (IJCAR 2012), pp. 286–300. doi:10.1007/978-3-642-31365-3_23.

Gao, S., Kong, S., Clarke, E.M., 2013. dReal: An SMT solver for nonlinear theories over the reals, in: Automated Deduction (CADE-24), pp. 208–214. doi:10.1007/978-3-642-38574-2_14.

Georgiev, B., Gómez-Serrano, J., Tao, T., Wagner, A.Z., 2025. Mathematical exploration and discovery at scale. arXiv:2511.02864.

Goldberg, D., 1991. What every computer scientist should know about floating-point arithmetic. ACM Computing Surveys 23, 5–48. doi:10.1145/103162.103163.

Google DeepMind, 2026. Formal conjectures: a collection of formalized conjectures in Lean. URL: https://github.com/google-deepmind/formal-conjectures.

Hales, T., Adams, M., Bauer, G., Dang, T.D., Harrison, J., Hoang, L.T., Kaliszyk, C., Magron, V., McLaughlin, S., Nguyen, T.T., Nguyen, Q.T., Nipkow, T., Obua, S., Pleso, J., Rute, J., Solovyev, A., Ta, T.H.A., Tran, N.T., Trieu, T.D., Urban, J., Vu, K., Zumkeller, R., 2017. A formal proof of the Kepler conjecture. Forum of Mathematics, Pi 5, e2. doi:10.1017/fmp.2017.1.

Higham, N.J., 2002. Accuracy and Stability of Numerical Algorithms. 2nd ed., SIAM, Philadelphia. doi:10.1137/1.9780898718027.

Hod, R., 2017. Improved lower bounds for the Fourier entropy/influence conjecture via lexicographic functions. URL: https://arxiv.org/abs/1711.00762, arXiv:1711.00762.

IEEE, 2018. IEEE standard for interval arithmetic (simplified). doi:10.1109/IEEESTD.2018.8277144. IEEE Std 1788.1-2017.

IEEE, 2019. IEEE standard for floating-point arithmetic. doi:10.1109/IEEESTD.2019.8766229. IEEE Std 754-2019.

Jaswal, M., 2026. Entropy certificate for the sums-differences constant C_{3c}: lower bound 1.674894211307116. doi:10.5281/zenodo.23289305. Zenodo deposit, version of 2026-10-10.

Jia, Y., Harman, M., 2011. An analysis and survey of the development of mutation testing. IEEE Transactions on Software Engineering 37, 649–678. doi:10.1109/TSE.2010.62.

Johansson, F., 2017. Arb: efficient arbitrary-precision midpoint-radius interval arithmetic. IEEE Transactions on Computers 66, 1281–1292. doi:10.1109/TC.2017.2690633.

Kapela, T., Mrozek, M., Wilczak, D., Zgliczyński, P., 2021. CAPD::DynSys: a flexible C++ toolbox for rigorous numerical analysis of dynamical systems. Communications in Nonlinear Science and Numerical Simulation 101, 105578. doi:10.1016/j.cnsns.2020.105578.

Katz, N.H., Tao, T., 1999. Bounds on arithmetic projections, and applications to the Kakeya conjecture. Mathematical Research Letters 6, 625–630. doi:10.4310/MRL.1999.v6.n6.a3.

Knight, J.C., Leveson, N.G., 1986. An experimental evaluation of the assumption of independence in multiversion programming. IEEE Transactions on Software Engineering SE-12, 96–109. doi:10.1109/TSE.1986.6312924.

Krawczyk, R., 1969. Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken. Computing 4, 187–201. doi:10.1007/BF02234767.

Lamb, C., Zacchiroli, S., 2022. Reproducible builds: Increasing the integrity of software supply chains. IEEE Software 39, 62–70. doi:10.1109/MS.2021.3073045.

Lean FRO, 2026. Comparator. URL: https://github.com/leanprover/comparator. Commit d03acab1.

Lin, Y., 2026. 147-point entropy certificate for the 4-slope sum-difference constant C_{3c}. URL: https://gist.github.com/CoolRmal/5368357cd781d7e5c676c9d68ad24d22. Gist, revision 62621d2d.

McConnell, R.M., Mehlhorn, K., Näher, S., Schweitzer, P., 2011. Certifying algorithms. Computer Science Review 5, 119–161. doi:10.1016/j.cosrev.2010.09.009.

Moore, R.E., Kearfott, R.B., Cloud, M.J., 2009. Introduction to Interval Analysis. SIAM, Philadelphia. doi:10.1137/1.9780898717716.

Mosaic Intelligence, 2026a. Certified sum-difference lower-bound certificates for C_{3a}, C_{3b}, and C_{3c}. doi:10.5281/zenodo.20794135. Zenodo deposit.

Mosaic Intelligence, 2026b. Improve upper bound for the de Bruijn–Newman constant to 0.1595 (certified record package). URL: https://github.com/teorth/optimizationproblems/pull/210. Pull request 210 to the optimization-constants registry; the bundle pinned at commit cc81078.

de Moura, L., Ullrich, S., 2021. The Lean 4 theorem prover and programming language, in: Automated Deduction – CADE 28, Springer. pp. 625–635. doi:10.1007/978-3-030-79876-5_37.

Necula, G.C., 1997. Proof-carrying code, in: Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 106–119. doi:10.1145/263699.263712.

Neumaier, A., 1990. Interval Methods for Systems of Equations. Cambridge University Press, Cambridge. doi:10.1017/CBO9780511526473.

Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.S., Wagner, A.Z., Shirobokov, S., Kozlovskii, B., Ruiz, F.J.R., Mehrabian, A., Kumar, M.P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., Balog, M., 2025. AlphaEvolve: a coding agent for scientific and algorithmic discovery. arXiv:2506.13131.

Numaro, 2026. Certificate archive: improved lower bound $C_{71} > 6.521845710923046575$ for the Fourier entropy-influence constant (n = 18 explicit construction). doi:10.5281/zenodo.21497769. Zenodo deposit.

O'Donnell, R., Tan, L.Y., 2013. A composition theorem for the Fourier entropy-influence conjecture. arXiv:1304.1347. ICALP 2013.

Ogita, T., Rump, S.M., Oishi, S., 2005. Accurate sum and dot product. SIAM Journal on Scientific Computing 26, 1955–1988. doi:10.1137/030601818.

OpenAI, 2026a. math: manuscripts and Lean challenges. URL: https://github.com/openai/math. Commit fd4aeeb2.

OpenAI, 2026b. NavierStokesAndEuler. URL: https://github.com/openai/NavierStokesAndEuler. Commits 8937a8f4 and f9e8bc5b.

Oxelra-AI, 2026. Qiushi-Engine-Kissing-Number-Research: the complete point sets. URL: https://github.com/Oxelra-AI/Qiushi-Engine-Kissing-Number-Research. Releases section-vectors-2026-10-11, configuration-vectors-2026-10-12 and lifting-vectors-2026-10-12, and issue #1; data CC BY 4.0.

Parrilo, P.A., 2003. Semidefinite programming relaxations for semialgebraic problems. Mathematical Programming 96, 293–320. doi:10.1007/s10107-003-0387-5.

Peyrl, H., Parrilo, P.A., 2008. Computing sum of squares decompositions with rational coefficients. Theoretical Computer Science 409, 269–281. doi:10.1016/j.tcs.2008.09.025.

Platt, D., Trudgian, T., 2021. The Riemann hypothesis is true up to $3 \cdot 10^{12}$. Bulletin of the London Mathematical Society 53, 792–797. doi:10.1112/blms.12460.

Prentice, R.L., 1974. A log gamma model and its maximum likelihood estimation. Biometrika 61, 539–544. doi:10.1093/biomet/61.3.539.

Raayoni, G., Gottlieb, S., Manor, Y., Pisha, G., Harris, Y., Mendlovic, U., Haviv, D., Hadad, Y., Kaminer, I., 2021. Generating conjectures on fundamental constants with the Ramanujan Machine. Nature 590, 67–73. doi:10.1038/s41586-021-03229-4.

rainrzk, 2026. erdos510-lambda4-audit: an independent re-certification of the $\lambda(4)$ argument with no shared code. URL: https://github.com/rainrzk/erdos510-lambda4-audit. Posted 2026-09-30 on teorth/erdosproblems issue #392.

Richardson, D., 1968. Some undecidable problems involving elementary functions of a real variable. Journal of Symbolic Logic 33, 514–520. doi:10.2307/2271358.

Rodgers, B., Tao, T., 2020. The de Bruijn–Newman constant is non-negative. Forum of Mathematics, Pi 8, e6. doi:10.1017/fmp.2020.6.

Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J.R., Ellenberg, J.S., Wang, P., Fawzi, O., Kohli, P., Fawzi, A., 2024. Mathematical discoveries from program search with large language models. Nature 625, 468–475. doi:10.1038/s41586-023-06924-6.

Rougier, N.P., Hinsen, K., Alexandre, F., Arildsen, T., Barba, L.A., Benureau, F.C.Y., et al., 2017. Sustainable computational science: the ReScience initiative. PeerJ Computer Science 3, e142. doi:10.7717/peerj-cs.142.

Rump, S.M., 1999. INTLAB — INTerval LABoratory, in: Csendes, T. (Ed.), Developments in Reliable Computing. Kluwer, Dordrecht, pp. 77–104. doi:10.1007/978-94-017-1247-7_7.

Rump, S.M., 2010. Verification methods: rigorous results using floating-point arithmetic. Acta Numerica 19, 287–449. doi:10.1017/S096249291000005X.

Rump, S.M., Zimmermann, P., Boldo, S., Melquiond, G., 2009. Computing predecessor and successor in rounding to nearest. BIT Numerical Mathematics 49, 419–431. doi:10.1007/s10543-009-0218-z.

Shewchuk, J.R., 1997. Adaptive precision floating-point arithmetic and fast robust geometric predicates. Discrete & Computational Geometry 18, 305–363. doi:10.1007/PL00009321.

Sibidanov, A., Zimmermann, P., Glondu, S., 2022. The CORE-MATH project, in: 2022 IEEE 29th Symposium on Computer Arithmetic (ARITH), pp. 26–34. doi:10.1109/ARITH54963.2022.00014.

Solovyev, A., Hales, T.C., 2013. Formal verification of nonlinear inequalities with Taylor interval approximations, in: NASA Formal Methods (NFM 2013), Springer. pp. 383–397. doi:10.1007/978-3-642-38088-4_26.

Tan, Y.K., Heule, M.J.H., Myreen, M.O., 2021. cake_lpr: Verified propagation redundancy checking in CakeML, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2021), pp. 223–241. doi:10.1007/978-3-030-72013-1_12.

Tao, T., 2026. Drop the unverified asterisk on the C_{3b} record. URL: https://github.com/teorth/optimizationproblems/pull/176#issuecomment-5848109776. The maintainer's comment of 2026-09-26 on pull request 176 to the optimization-constants registry, which another contributor opened under this title; the comment states the rule that the registry's contributing guide then carried.

Tao, T., et al., 2026. A collection of optimization problems in mathematics. URL: https://github.com/teorth/optimizationproblems. The optimization-constants registry and its contributing guide; read at commits 2c1968cd (2026-09-26), ec37689f and 46dbf036 (2026-10-18), pinned in the repository.

The mathlib Community, 2020. The Lean mathematical library, in: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM. pp. 367–381. doi:10.1145/3372885.3373824.

Toledo, C., 2026a. cert-machine: independent exact certification of machine-generated mathematics. doi:10.5281/zenodo.22225860. The repository archive: code, records and reports. The concept DOI resolves to the latest version; the version this paper was built from is named on its title page.

Toledo, C., 2026b. Certified extreme-value analysis of significant wave height: maximum-likelihood fits and return levels over a public global hindcast, and environmental contours decided exactly. URL: https://carlostoledo.co/paper/p1-return-levels.pdf. Archived with the repository, doi:10.5281/zenodo.22225860. Draft v0.2.

Toledo, C., 2026c. A decision procedure for machine-proposed engineering numbers: three-valued verdicts over a declared uncertainty envelope, with witnesses and the threshold that decides. URL: https://carlostoledo.co/paper/p2-decision-procedure.pdf. Archived with the repository, doi:10.5281/zenodo.22225860. Draft.

Toledo, C., 2026d. Independent exact certification of machine-generated mathematics: the first register. URL: https://carlostoledo.co/paper/register.pdf. Archived with the repository, doi:10.5281/zenodo.22225860. Draft.

Toledo, C., 2026e. The twelve-block colouring is optimal: $\delta_3 = 117/2192$ — a computer-assisted proof, its certificates re-decided by two programs that share no code. URL: https://carlostoledo.co/paper/delta3.pdf. Archived with the repository, doi:10.5281/zenodo.22225860. Draft; the certificates are archived with the repository release, doi:10.5281/zenodo.23171167.

Toledo, C., 2026f. The value of $\lambda(4)$: a machine-derived proof completing Mercer's Section-5 program. doi:10.5281/zenodo.22225861. Report at https://carlostoledo.co/reports/lambda4.html.

Tucker, W., 2002. A rigorous ODE solver and Smale's 14th problem. Foundations of Computational Mathematics 2, 53–117. doi:10.1007/s002080010018.

Wetzler, N., Heule, M.J.H., Hunt, W.A., 2014. DRAT-trim: Efficient checking and trimming using expressive clausal proofs, in: Theory and Applications of Satisfiability Testing – SAT 2014, pp. 422–429. doi:10.1007/978-3-319-09284-3_31.

Yang, S., Zhao, R., Wu, J., Wang, Y., Chen, F., Zhu, K., Li, W., Li, Z., Li, Y., Hong, S., Pan, Y., Yang, J., Deng, T., Mi, J., Chen, H., Yang, Y., 2026. Large-scale autonomous discovery of kissing number constructions. arXiv:2609.35051.

Yap, C.K., 1997. Towards exact geometric computation. Computational Geometry 7, 3–23. doi:10.1016/0925-7721(95)00040-2.